

Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

Sta. Mesa, Manila

LAlgo

Language Design Specification

A Domain-Specific Language for Data Structures and Algorithms

Lexical Specification and EBNF Syntax Proposal

Submitted by: Group 4

Buenaventura, Harvey L.

De Mesa, Joshua C.

Dela Rosa, Mico Von Adriel A.

Magalona, Adriel M.

Morales, Timothy Gabriel

Nevado, Sean Andrew M.

Salvacion, Dean James G.

Submitted to

Prof. Ria A. Sagum

Course and Subject Code

Principles of Programming Languages (COSC 303)

12 November 2026

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Inspiration . . . 1
- 1.3 Problem Addressed . . . 1
- 1.4 Objectives . . . 1

2 Lexical Elements **1**
- 2.1 Character Set . . . 1
- 2.2 Identifiers . . . 2
- 2.3 Finite Automata (Lexical Analyzer Execution Model) . . . 2
  - 2.3.1 Unified DFA with One Start State . . . 3
  - 2.3.2 DFA for Identifiers and Keywords . . . 5
  - 2.3.3 DFA for Numeric Literals (Integer and Decimal) . . . 5
  - 2.3.4 DFA for Relational and Assignment Operators . . . 6
  - 2.3.5 DFA for Comments and Division . . . 6
  - 2.3.6 DFA for String Literals . . . 7
  - 2.3.7 DFA for Character Literals . . . 7
  - 2.3.8 DFA for Whitespace (Blanks) . . . 7
  - 2.3.9 DFA for Delimiters and Brackets . . . 8
  - 2.3.10 Representative DFA for Keyword Recognition . . . 8
  - 2.3.11 Scope and Limitations . . . 8
  - 2.3.12 Token Structure and Error Handling . . . 9
  - 2.3.13 Implementation Input, Output, and Demonstration . . . 10
- 2.4 Operation Symbols . . . 11
  - 2.4.1 Arithmetic Operators . . . 11
  - 2.4.2 Boolean (Logical) Operators . . . 12
  - 2.4.3 Relational Comparison Operators . . . 12
- 2.5 Keywords and Reserved Words . . . 13
- 2.6 Noise Words . . . 13
- 2.7 Comments . . . 13
  - 2.7.1 Single-Line Comments . . . 14
  - 2.7.2 Multi-Line Comments . . . 14
- 2.8 Blanks (Whitespace Handling) . . . 14
- 2.9 Delimiters and Brackets . . . 14
- 2.10 Free-Field Format . . . 15
- 2.11 Expressions . . . 15
  - 2.11.1 Operator Precedence and Associativity . . . 16
- 2.12 Primitive Data Types . . . 16
  - 2.12.1 Integer . . . 16
  - 2.12.2 Decimal . . . 16
  - 2.12.3 Character . . . 17
  - 2.12.4 Boolean . . . 17
  - 2.12.5 Collection Types . . . 17
- 2.13 From Lexical Tokens to Statements . . . 17

3 Syntax Grammar Rules Proposal (EBNF) **18**
- 3.1 Grammar contract . . . 18
- 3.2 Reading the EBNF . . . 18
- 3.3 Productions . . . 18
- 3.4 Statement coverage . . . 20
- 3.5 Parsing approach and token handoff . . . 20
- 3.6 Syntax errors and panic recovery . . . 20
- 3.7 One example, two derivations, and one parse tree per rule . . . 21

A Worked Binary-Search Token Stream **45**

B Complete Unified DFA Transition Specification **47**
- B.1 All Transitions from the Single Start State . . . 47
- B.2 All States, Accepting Actions, and Remaining Transitions . . . 48

1 Introduction

1.1 Overview

LAlgo is a proposed imperative domain-specific language (DSL) for expressing data structures and algorithms in readable pseudocode. The unified C lexer implements lexical recognition. Chapter 2 documents its token rules, and Chapter 3 proposes an EBNF grammar and a top-down syntax analyzer.

The proposed language includes declarations, input and output, assignment, conditionals, and iteration. It uses one loop kind, `WHILE`, in single and nested forms. A binary-search program illustrates how these constructs work together.

1.2 Inspiration

Algorithm descriptions in textbooks use familiar steps, conditions, and loop guards. LAlgo keeps that vocabulary close to the source and uses explicit statement and block boundaries so readers can follow the order of operations.

1.3 Problem Addressed

Pseudocode conventions vary across textbooks. A reader may have to infer where a statement ends, how a condition groups its operators, or which branch belongs to a conditional. LAlgo addresses these questions with a defined token vocabulary and an explicit grammar. The binary-search example illustrates how array access, comparison, assignment, and iteration fit into one program.

1.4 Objectives

1. Specify the character alphabet, token categories, and unified lexical automaton.
2. Preserve token spellings and source positions for syntax analysis and diagnostics.
3. Define the grammar for declarations, input and output, assignments, selection, and a single loop kind with nesting.
4. Provide one example per named grammar rule and show its leftmost and rightmost expansion order.
5. Specify a top-down parser design with a sequential terminal handoff and recovery after syntax errors.

2 Lexical Elements

2.1 Character Set

LAlgo utilizes a standard ASCII-based character set denoted by the finite alphabet Σ. The alphabet is organized into lexical classes:

$$\begin{aligned}\Sigma = \Sigma_{\text{alpha}} \cup \Sigma_{\text{digits}} \cup \Sigma_{\text{arithmetic}} \cup \Sigma_{\text{relational}} \cup \Sigma_{\text{delimiters}} \\ \cup \Sigma_{\text{underscore}} \cup \Sigma_{\text{quotes}} \cup \Sigma_{\text{whitespace}} \cup \Sigma_{\text{literal}} \cup \Sigma_{\text{comment}}.\end{aligned}$$

The supported character categories are explicitly defined below.

Alphabetic Characters (Σ_{alpha}) `a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`

Numeric Digits (Σ_{digits}) `0 1 2 3 4 5 6 7 8 9`

Arithmetic Symbols ($\Sigma_{\text{arithmetic}}$) `+ - * / % ^`

Relational and Assignment Symbols ($\Sigma_{\text{relational}}$) `= ! < >`

Delimiters and Punctuation ($\Sigma_{\text{delimiters}}$) `( ) [ ] { } , ; : .`

Identifier Continuation Symbol ($\Sigma_{\text{underscore}}$) `_`

Quotation Symbols (Σ_{quotes}) `" '`

Whitespace Characters ($\Sigma_{\text{whitespace}}$) Space (ASCII 32), horizontal tab (ASCII 9), and line feed (ASCII 10).

Literal/Comment-Only Symbols (Σ_{literal}) Printable ASCII 33–126 not already in another class and other than backslash; these symbols are permitted only inside quoted literals and comments.

Comment-Only Symbol (Σ_{comment}) Backslash (ASCII 92), permitted in comments only; it does not introduce an escape.

Space and horizontal tab are also permitted inside string literals, character literals, and comments, even though they belong to $\Sigma_{\text{whitespace}}$.

The scanner combines the individual relational characters into `==`, `!=`, `<=`, and `>=` using lookahead. These are multi-character operators rather than additional members of the character alphabet; `!` alone is not a valid operator.

2.2 Identifiers

Identifiers are programmer-defined tokens used to name variables, constants, and algorithmic routines. An identifier must satisfy the following lexical constraints:

- It must begin with an alphabetic character (`a-z` or `A-Z`).
- It may subsequently contain alphabetic characters, numeric digits (`0-9`), or underscores (`_`).
- It must not match any reserved keyword or noise word.
- It is strictly case-sensitive: `counter`, `Counter`, and `COUNTER` are distinct tokens.

Formally, the lexical structure of an identifier is expressed by the regular expression:

```
[a-zA-Z][a-zA-Z0-9_]*
```

Valid identifiers

```
low
high
binarySearch
target_index
arraySize2
```

Invalid identifiers

```
2low // Lexical error: number followed by
    letter
target-index // Hyphen means subtraction
WHILE        // Reserved keyword
SET          // Reserved noise word
```

2.3 Finite Automata (Lexical Analyzer Execution Model)

LAlgo's implemented lexical scanner processes raw source code using one unified deterministic finite automaton (DFA), with one start state and one shared transition table. Formally, the automaton is defined as a five-tuple:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where Q is the finite set of internal states, Σ is the input character alphabet, $\delta : Q \times \Sigma \to Q$ is the state transition function, $q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.

Diagrams show only defined transitions; every missing transition goes to an implicit non-accepting error state, so δ is total. State sets listed below omit this implicit state for readability. The scanner remembers the last accepting position and leaves lookahead for the next token. Entering the error state ends a candidate match; it reports an error only if no valid token can be emitted or an explicit lexical restriction is violated. Once a quote or comment opener is recognized, an unterminated construct is an error, not a fallback to a shorter token.

2.3.1 Unified DFA with One Start State

The implementation in `lalgo_unified.c` builds one table, `DELTA[state][class]`, with 254 explicit states and 83 input classes. Every candidate lexeme starts at the same q_0. Keywords, identifiers, numeric literals, operators, comments, quoted literals, delimiters, and whitespace are paths within this single state graph. There is no runtime dispatch to separate recognizers and no post-scan string comparison for keywords.

The figure below is a grouped overview of the unified graph, not a substitute for its complete transitions. Each box denotes a state family; keyword letters and individual punctuation have distinct concrete states. Appendix B gives every state, accepting action, and defined transition directly from the C implementation. The category diagrams that follow use local state labels as explanatory projections of these paths; they are not independently executed DFAs and their local q_0 denotes the same global start.

After each transition the scanner remembers the last accepting state and source position. It emits the longest accepted prefix, retains unused lookahead, then restarts at q_0 for the next lexeme. Whitespace accepts with a skip action. A malformed-number state accepts an INVALID candidate for diagnostic reporting. Unterminated quoted literals and block comments use committed-state metadata to prevent fallback to a shorter token. These scanner actions are distinct from DFA transitions.

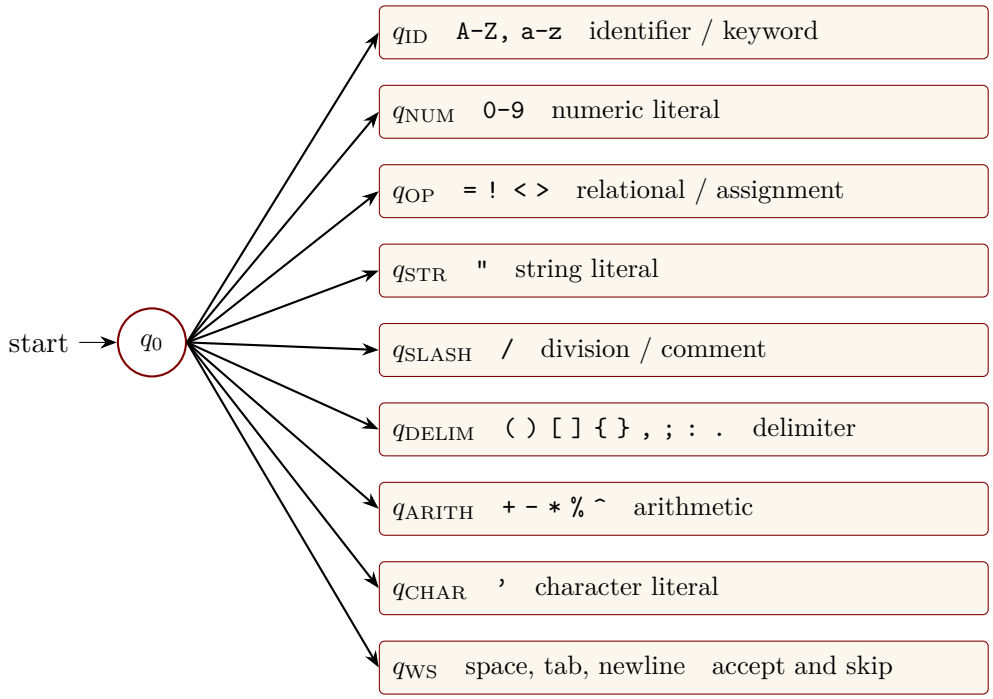


Figure 1: Grouped overview of one unified DFA: all token families branch from the single start state q_0.

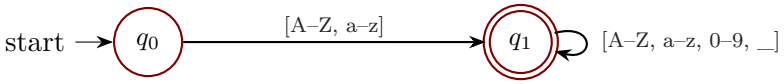
Step-by-step lexical scan trace. Consider the statement `count = 10`.

Step	Current state	Character(s)	Automaton action	Token emitted
1	q_0	`'c'`	Transition to q_{ID}	None
2	q_{ID}	`'o', 'u', 'n', 't'`	Self-loop on q_{ID}	None
3	q_{ID}	space	No accepting extension; emit and reset	`TOKEN(IDENT, "count")`
4	q_0	space	Consume whitespace recognizer	None
5	q_0	`'='`	Transition to q_{OP}	None
6	q_{OP}	space	Lookahead is not `'='`; emit and reset	`TOKEN(ASSIGN, "=")`
7	q_0	space	Consume whitespace recognizer	None
8	q_0	`'1'`	Transition to q_{NUM}	None
9	q_{NUM}	`'0'`	Self-loop on q_{NUM}	None
10	q_{NUM}	EOF	Buffer end; emit and reset	`TOKEN(INTEGER, 10)`

2.3.2 DFA for Identifiers and Keywords

Identifiers begin with an alphabetic character and may continue with letters, digits, or underscores. This diagram projects the general identifier shape. In the full unified DFA, uppercase keyword-prefix states accept their associated token at complete spellings; a non-keyword continuation falls into the general identifier state. Lowercase or mixed-case prefix states accept IDENT and flag a complete keyword spelling as suspected. Keyword recognition is part of the same scan, not a later lookup.

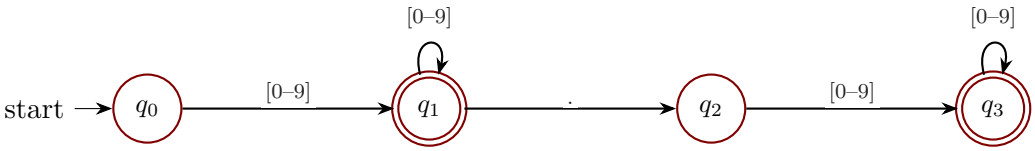
- $Q = \{q_0, q_1\}$
- q_0: initial state
- $F = \{q_1\}$ (accepting state)



2.3.3 DFA for Numeric Literals (Integer and Decimal)

This automaton distinguishes whole-number values (**INTEGER**) from fractional real numbers (**DECIMAL**) through decimal-point transitions.

- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_1, q_3\}$, where q_1 accepts **INTEGER** and q_3 accepts **DECIMAL**

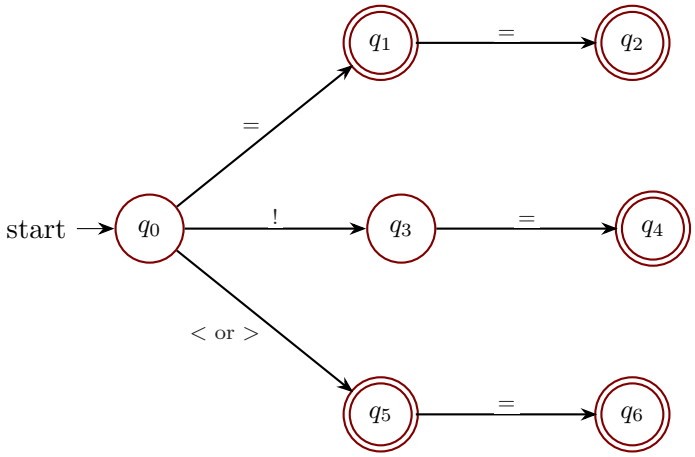


- **Trailing dot.** `3.` is not a `DECIMAL`. From non-accepting q_2, the scanner backtracks to the last accepting position and emits `INTEGER 3`, followed by `DOT`.
- **Leading dot.** `.5` is `DOT` followed by `INTEGER 5`. A decimal requires at least one digit on each side of the point.
- **Leading zeros.** Allowed and ignored in value: `007` = `7` and `00.5` = 0.5. The original spelling remains in the token's lexeme.
- **Range.** Integer literals must fit the signed 32-bit range; their maximum unsigned magnitude is 2147483647. Larger literals are lexical errors. Since signs are separate operators, even the magnitude 2147483648 in `-2147483648` is rejected. Double precision is the proposed runtime representation for decimals; this lexer recognizes decimal spelling without converting values or checking floating-point range.

2.3.4 DFA for Relational and Assignment Operators

This DFA disambiguates single-character assignment and comparison operators (`=`, `<`, `>`) from compound relational operators (`==`, `!=`, `<=`, `>=`).

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$
- $F = \{q_1, q_2, q_4, q_5, q_6\}$

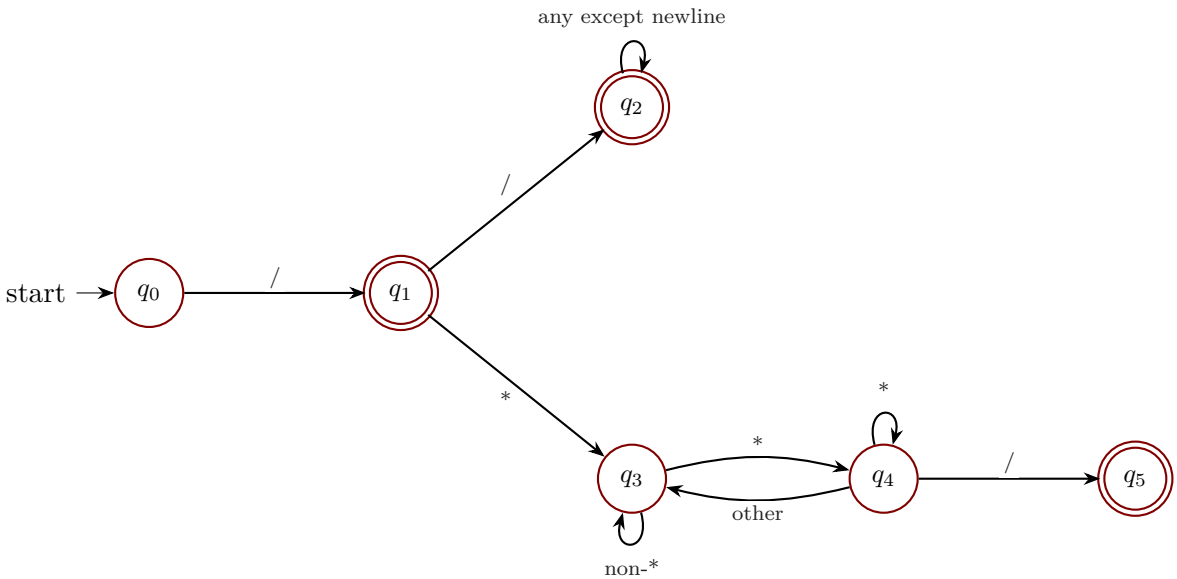


2.3.5 DFA for Comments and Division

Transition labels such as “other” and “non-*” range only over the permitted comment-body characters in Section 2.7; unsupported characters are lexical errors.

This DFA distinguishes division (`/`) from single-line comments (`//`) and block comments (`/* ... */`) using lookahead scanning.

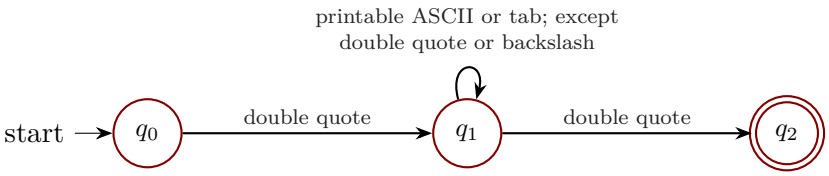
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
- $F = \{q_1, q_2, q_5\}$, where q_1 accepts division, q_2 accepts a single-line comment, and q_5 accepts a multi-line comment



2.3.6 DFA for String Literals

This DFA recognizes character-string literals enclosed by double quotation marks (`"..."`). Their contents consist of printable ASCII characters other than a double quote or backslash, plus horizontal tab; escape sequences and embedded newlines are not defined.

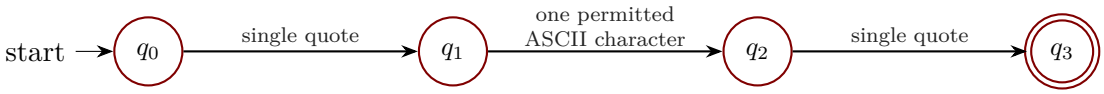
- $Q = \{q_0, q_1, q_2\}$
- $F = \{q_2\}$



2.3.7 DFA for Character Literals

A character literal consists of one printable ASCII character or horizontal tab enclosed in single quotation marks, such as `'A'` or `','`. The character between the quotes may be ASCII 32–126 except a single quote or backslash, or a horizontal tab; escape sequences are not defined in this specification. An empty, multi-character, or unterminated literal is a lexical error.

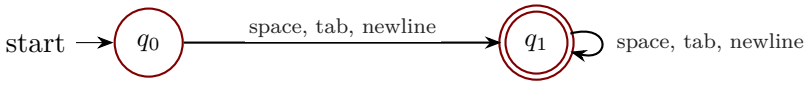
- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_3\}$



2.3.8 DFA for Whitespace (Blanks)

Whitespace characters—spaces, horizontal tabs, and line feeds—are recognized by a trivial single-loop automaton. Tokens matched by this automaton are consumed by the lexer but are not emitted to the parser; they separate adjacent lexemes and update source positions and newline metadata.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$ (accepting, but discarded)

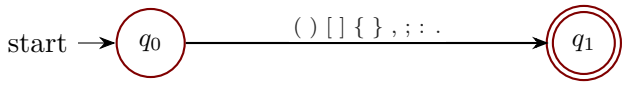


Because whitespace carries no semantic weight (Section 2.8), the scanner discards any accepted whitespace lexeme instead of forwarding it to the parser, preserves any line-feed information in token metadata, then resumes scanning from q_0.

2.3.9 DFA for Delimiters and Brackets

Delimiters and brackets are single-character tokens: `( ) [ ] { } , ; : .` . Because no lookahead is required, each is accepted after exactly one transition from q_0.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$, where the specific delimiter determines the emitted token type (for example, `LPAREN`, `RBRACE`, or `COMMA`)



Unlike the multi-character automata above, no maximal munch is needed: the lexer emits the corresponding token immediately after the single accepted transition and resets to q_0.

2.3.10 Representative DFA for Keyword Recognition

The identifier DFA defines the shape shared by identifiers and reserved words: a letter followed by zero or more letters, digits, or underscores. The automaton below illustrates how that general shape narrows into the exact spelling of **IF**, **INPUT**, **INTEGER**, and **INVARIANT**. The first three are current language keywords; **INVARIANT** is reserved for a future extension. They share the prefix `I` and diverge as their spellings differ.

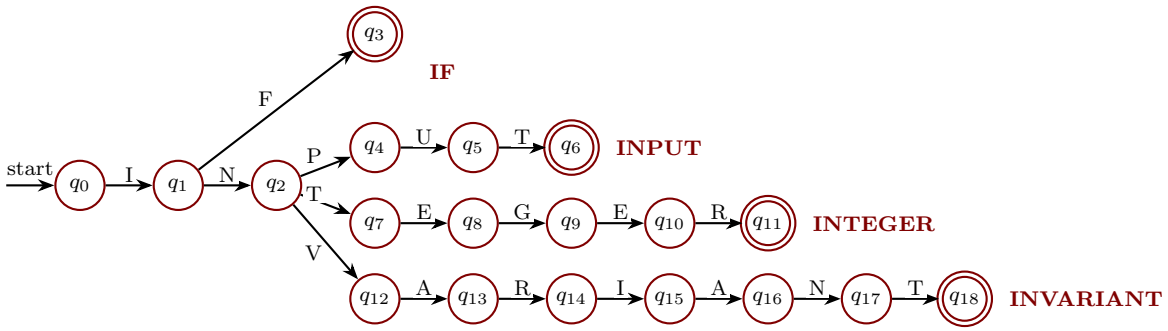


Figure 2: Shared-prefix keyword recognition with non-overlapping branches and explicit accepting states.

At each accepting state (q_3, q_6, q_{11}, and q_{18}), if the following character continues the identifier pattern `[a-zA-Z0-9_]` instead of a delimiter or whitespace, the scanner does not emit a keyword token. It falls back into the general identifier state q_{ID}. For example, `IFrame` does not match **IF** because `r` extends the identifier. Any unshown letter at a branch likewise routes to q_{ID}.

> **Note.** The same shared-prefix principle extends to other current keywords, future-reserved words, and noise words. For example, **ALGORITHM**, **ARRAY**, and **AND** share an A prefix, while **TRACE**, **TRUE**, and **THE** share a T prefix. The keyword trie is used only when constructing the unified table. During scanning, keyword letters are transitions in that same table. Exact-case states and mixed-case states are listed exhaustively in Appendix B; this diagram shows a representative uppercase projection.

2.3.11 Scope and Limitations

This finite-automata model operates strictly at the lexical level, where tokens are recognized and classified. It does not check type rules, declaration-before-use, statement structure, or grammar-level correctness. Those checks belong to parsing and semantic analysis.

Several simplifications keep the model clear and within the scope of lexical analysis:

- **Error handling.** The error state is implicit. An unterminated string literal, character literal, or block comment ends in a non-accepting state and is reported as a lexical error.
- **Keyword resolution.** The keyword diagram is a projection of the unified table. Exact uppercase prefixes and prefixes containing lowercase letters have separate states. Digits, underscores, and non-matching letters lead to the general identifier state; no separate runtime keyword lookup is performed.
- **Future-reserved words.** `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` are recognized as reserved lexemes but have no grammar or runtime semantics in this iteration.
- **Nested brackets.** At the lexical level, `{`, `}`, `(`, `)`, `[`, and `]` are individual delimiter tokens. Nesting and matching are parser responsibilities.
- **Newlines.** CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted.
- **Block comments.** Block comments do not nest; the first `*/` closes the comment.
- **Quotes inside literals.** Because escape sequences are not defined, a string cannot contain a double quote and a character literal cannot contain a single quote. Backslash is not permitted in either literal; escapes are future work.
- **Signed numbers.** The lexer emits a minus operator separately from the unsigned literal. Chapter 3 assigns unary minus its role in expressions.
- **Number-identifier boundaries.** A numeric literal immediately followed by an alphabetic character or underscore (for example, `5thSample`, `2low`, or `2_low`) is a lexical error: `Malformed number '2low'`. The scanner consumes the numeric candidate and the entire following run of `[A-Za-z0-9_]` as one bad lexeme and resumes after it. For example, `1.5abc` is one malformed number.

2.3.12 Token Structure and Error Handling

Every token is a record `(type, lexeme, line, column, precededByNewline)` with an additional Boolean `suspectedKeyword` flag (default `FALSE`). The lexeme preserves the exact source spelling, including quotation marks for quoted literals. Lines and columns start at 1 and identify the beginning of the token; a tab counts as one column. Positions refer to the source after CRLF normalization. EOF has an empty lexeme and the position immediately after the final source character.

Class	Token types
Identifier	IDENT
Literals	INTEGER, DECIMAL, STRING, CHAR, TRUE, FALSE
Type keywords	KW_INTEGER, KW_DECIMAL, KW_CHARACTER, KW_BOOLEAN, KW_ARRAY, KW_STRING
Control/I/O keywords	KW_IF, KW_ELSE, KW_WHILE, KW_INPUT, KW_OUTPUT
Legacy lexical token	KW_FOR (excluded from Chapter 3)
Operator keywords	DIV, AND, OR, NOT
Arithmetic	PLUS, MINUS, STAR, SLASH, PERCENT, CARET
Relational / assignment	ASSIGN, EQ, NEQ, LT, GT, LE, GE
Delimiters	LPAREN, RPAREN, LBRACKET, RBRACKET, LBRACE, RBRACE, COMMA, SEMICOLON, COLON, DOT
Noise	NOISE_SET, NOISE_THE
Future-reserved	RES_ALGORITHM, RES_RETURN, RES_TRACE, RES_INVARIANT
Comments	LINE_COMMENT, BLOCK_COMMENT
Invalid	INVALID
Special	EOF

Whitespace produces no token. Comments receive their own token entries (LINE_COMMENT and BLOCK_COMMENT). Invalid quoted literals, unterminated block comments, malformed numbers, out-of-range integers, illegal standalone characters, and lone `!` produce INVALID entries with line-and-column diagnostics. A terminated comment containing unsupported bytes retains its comment classification and receives diagnostics at those byte positions. Scanning continues after recovery; EOF is the last token for a source that passes input validation. Recovery uses these boundaries:

- Malformed numbers consume the whole bad numeric run described in Section 2.3.11; out-of-range integers consume their entire digit sequence.
- Invalid quoted literals consume through the matching closing quote, or stop before a line feed or at EOF if no closing quote occurs first. The next line remains available for scanning.
- An invalid comment consumes through its normal terminator (line feed/EOF for a line comment, first `*/` for a block comment). An unterminated block comment consumes to EOF. Unsupported characters inside comments are reported at their own positions.
- An illegal standalone character or lone `!` consumes one character. Skipped text still updates positions and newline metadata.

The earlier scan trace abbreviates token records. For `count = 10` on one line with no trailing newline, the complete records are:

```
(IDENT,   "count", 1,  1, FALSE)
(ASSIGN,  "=",     1,  7, FALSE)
(INTEGER, "10",    1,  9, FALSE)
(EOF,     "",      1, 11, FALSE)
```

Here and in the worked example, `suspectedKeyword` is `FALSE` unless explicitly stated.

2.3.13 Implementation Input, Output, and Demonstration

The executable accepts a source filename ending in `.lal` (case-insensitive extension); other extensions are rejected before scanning. It also rejects files containing NUL bytes. This is an input check, not a

guarantee that renamed binary data is valid source: unsupported bytes are still subject to lexical diagnostics. Keywords remain case-sensitive regardless of the extension's case.

```
gcc -std=c99 -Wall -Wextra -o lalgo lalgo_unified.c
./lalgo sample.lal
./lalgo sample.lal t
```

The optional `t` enables a character-by-character unified DFA trace; normal operation prints tokens and errors without tracing. Console columns are line:column, token type, lexeme, newline metadata, and suspected-keyword metadata. The scanner writes `Symbol Table.txt` in the current working directory, replacing the previous output. Each non-EOF occurrence receives an entry containing its number, lexeme, token type, classification, and source position. Comments and INVALID entries are included. This required output is a lexical occurrence listing, not a declaration table with variable types, scopes, addresses, or runtime values. Comment and invalid displays escape line feeds and tabs and truncate long text; internal lexemes retain the source spelling.

Exit status 0 means no lexical errors, 1 means lexical errors were recorded, and 2 indicates usage or file input/output failure. Lexical success does not establish grammatical validity or execute the program.

2.4 Operation Symbols

LAlgo defines seven arithmetic operators, three core logical operators, and six relational comparison operators. The descriptions below indicate intended meanings; they do not constitute a runtime implementation.

2.4.1 Arithmetic Operators

LAlgo provides seven foundational arithmetic operations. `^` denotes exponentiation, following mathematical and textbook pseudocode notation; unlike C/Java, it is not bitwise XOR:

Operation	Operator	Associativity	Semantic description
Addition	`+`	Left	Computes the sum of operands.
Subtraction	`-`	Left	Computes the difference of operands.
Multiplication	`*`	Left	Computes the product of operands.
Division	`/`	Left	Computes a real quotient (`DECIMAL`).
Modulo	`%`	Left	Remainder after integer division.
Integer division	`DIV`	Left	Returns the truncated integer quotient.
Exponentiation	`^`	Right	Raises the base to a power.

Distinction between / and DIV:

```
INTEGER a = 7;
INTEGER b = 2;
DECIMAL quotient = a / b;       // 3.5
INTEGER truncated = a DIV b;    // 3
INTEGER remainder = a % b;      // 1
```

Exponentiation example:

```
INTEGER power = 2 ^ 3; // 8
```

2.4.2 Boolean (Logical) Operators

Boolean expressions use three logical keywords. This lexical specification identifies their tokens; evaluation strategy, including any short-circuit behavior, is outside its scope.

Operation	Operator	Precedence	Semantic description
Logical NOT	`NOT`	Highest among logical operators	Unary negation; inverts the truth value.
Logical AND	`AND`	Medium	Conjunction; `TRUE` only if both operands are `TRUE`.
Logical OR	`OR`	Lowest	Disjunction; `TRUE` if at least one operand is `TRUE`.

2.4.3 Relational Comparison Operators

Comparison operators evaluate the quantitative relationship between expressions and yield a strictly typed `BOOLEAN` value (`TRUE` or `FALSE`).

Comparison	Symbol	Example
Equal to	`==`	`mid == target`
Not equal to	`!=`	`count != 0`
Less than	`<`	`low < high`
Greater than	`>`	`score > 75`
Less than or equal to	`<=`	`i <= length`
Greater than or equal to	`>=`	`val >= 0`

2.5 Keywords and Reserved Words

Reserved words are recognized by the lexer and cannot be used as identifiers. The current language keywords and the words held for future extensions are distinguished below. All use uppercase canonical spelling. Keywords are recognized only in their uppercase spelling. An identifier whose lowercase form equals the lowercase form of a reserved word (for example, `while` or `If`) is emitted as `IDENT` with `suspectedKeyword` set to `TRUE`, so later phases can report "Did you mean WHILE?". Such an identifier is not a lexical error and is not reserved. The same diagnostic rule applies to legacy, future-reserved, and noise words. Identifiers such as `input`, `string`, and `array` remain legal.

Category	Reserved words
Primitive types	`INTEGER`, `DECIMAL`, `CHARACTER`, `BOOLEAN`
Collection types	`ARRAY`, `STRING`
Control flow	`IF`, `ELSE`, `WHILE`
Legacy lexical spelling	`FOR` (excluded from Chapter 3)
I/O statements	`INPUT`, `OUTPUT`
Boolean literals	`TRUE`, `FALSE`
Arithmetic operator	`DIV`
Logical operators	`AND`, `OR`, `NOT`
Future extensions (reserved only)	`ALGORITHM`, `RETURN`, `TRACE`, `INVARIANT`

The lexer recognizes 17 current reserved spellings, one legacy spelling (`FOR`), and four future-reserved words. The grammar uses the current spellings and has no production for the legacy or future-reserved forms. The separate noise words `SET` and `THE` are also reserved.

2.6 Noise Words

Noise words are optional lexical tokens designed to mirror natural-language pseudocode. They enhance readability for beginner programmers but carry no operational weight. Chapter 3 permits these words only in the positions shown below. The lexer recognizes them at any token boundary. They remain reserved to prevent identifier ambiguity.

- `SET`: optional prefix before an assignment.

```
SET index = 0; // Equivalent to: index = 0;
```

- `THE`: optional readability marker in output routines.

```
OUTPUT THE result; // Equivalent to: OUTPUT result;
```

2.7 Comments

Comments allow programmers to document algorithmic logic, state specifications, and preconditions. The implemented scanner recognizes comments and includes them in its console and Symbol Table output. The proposed parser filters these comment records before reading grammar terminals. Comment bodies may contain any printable ASCII character (32–126), including backslash, and horizontal tab. Block comments may also contain line feeds. Any other character inside a comment is a lexical error. The regular expressions below describe comment boundaries subject to these character restrictions.

2.7.1 Single-Line Comments

Single-line comments begin with two forward slashes (`//`) and continue until the next newline.

Regular expression:

```
//[^\n]*
```

Example:

```
// Initialize the lower search boundary
INTEGER low = 0
```

2.7.2 Multi-Line Comments

Multi-line comments are delimited by `/*` and `*/`, allowing block documentation across multiple lines.

Regular expression:

```
/\*[\s\S]*?\*/
```

Example:

```
/*
Binary Search Implementation
Precondition: Input array must be sorted in ascending order.
Returns: Index of target element, or -1 if not found.
*/
```

2.8 Blanks (Whitespace Handling)

Whitespace includes spaces (ASCII 32), horizontal tabs (ASCII 9), and line feeds (ASCII 10). CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted. LAlgo treats whitespace strictly as a non-semantic token separator:

- Whitespace demarcates adjacent keywords, identifiers, and literals.
- Consecutive whitespace characters collapse into one lexical boundary.
- Indentation carries no structural or scoping significance, preventing syntax errors caused by mixed tabs and spaces.

These lines are lexically equivalent:

```
INTEGER x = 100;
```

```
INTEGER       x    =    100;
```

2.9 Delimiters and Brackets

Delimiters mark statement boundaries, collection bounds, grouping, and blocks in the proposed syntax.

- **Parentheses `( )`:** enclose grouped expressions and the guards of conditionals and loops.

- **Square brackets `[ ]`:** delimit array literals and specify indexed element access.
- **Curly braces `{ }`:** delimit the statement blocks of conditionals and loops.
- **Comma `,`:** separates declaration names and array elements and divides the two operands of mixed output.
- **Semicolon `;`:** ends a declaration, input, output, or assignment statement in Chapter 3.

2.10 Free-Field Format

LAlgo uses a free-field format. Program source code is not bound to fixed columns, physical margins, or indentation rules.

- **Block markers.** Braces delimit blocks. Chapter 3 defines how blocks and statements can nest.
- **Statement formatting.** Outside comments and quoted literals, spaces, tabs, and line feeds separate tokens. The punctuation and productions in Chapter 3 determine statement boundaries.

Line feeds are consumed and never emitted as separate tokens. For ordinary tokens and EOF, `precededByNewline` records whether a line feed occurred since the preceding ordinary token (or the start of the source), including within comments and recovered invalid text. Comment and INVALID entries are auxiliary: their own flag records a line feed since the preceding visible entry, but emitting them does not clear the pending flag for the next ordinary token. Thus the next ordinary token retains its newline information even when a comment is listed between statements. The parser design uses explicit terminators for statement boundaries and retains the metadata for diagnostics.

Illustrative free-field source:

```
INTEGER low = 0; INTEGER high = 10;
IF (low < high) { OUTPUT low; }
```

2.11 Expressions

The examples below show token sequences for intended arithmetic, relational, and logical expressions. Chapter 3 specifies their grammar and grouping. Type checking and evaluation belong to later stages.

1. **Arithmetic expressions** evaluate to numeric types (`INTEGER` or `DECIMAL`).

   ```
   (low + high) DIV 2
   ```

2. **Relational expressions** compare values to produce a truth condition.

   ```
   numbers[mid] > target
   ```

3. **Logical expressions** combine Boolean expressions using propositional connectives.

   ```
   (low <= high) AND (found == FALSE)
   ```

2.11.1 Operator Precedence and Associativity

Rank	Operator	Description	Associativity
1	`( ) [ ]`	Grouping and array indexing	Left-to-right
2	`^`	Exponentiation	Right-to-left
3	`-` (unary)	Sign inversion	Right-to-left
4	`* / %`, `DIV`	Multiplicative operations	Left-to-right
5	`+ -`	Additive operations	Left-to-right
6	`< <= > >=`	Relational inequalities	Left-to-right
7	`== !=`	Relational equalities	Left-to-right
8	`NOT`	Logical negation	Right-to-left
9	`AND`	Logical conjunction	Left-to-right
10	`OR`	Logical disjunction	Left-to-right

The table lists intended precedence from highest to lowest. In particular, `NOT found == TRUE` groups as `NOT (found == TRUE)`. Chapter 3 incorporates these grouping rules into the syntax proposal.

Operator choices compared with C and Java

Operation	C and Java	LAlgo design
Integer operands in division	`/` truncates the quotient toward zero when it is representable.	`DIV` requests the truncated integer quotient. `/` requests a decimal quotient.
Caret	`^` applies bitwise XOR to integer operands.	`^` denotes right-associative exponentiation.
Logical spelling	`!`, `&&`, and `||`.	`NOT`, `AND`, and `OR` follow pseudocode spelling.

References: WG14 N1570, Sections 6.5.3.3, 6.5.5, and 6.5.11–6.5.14. Java Language Specification, Chapter 15, Sections 15.15.6, 15.17.2, and 15.22–15.24.

2.12 Primitive Data Types

LAlgo supports four fundamental primitive types and two built-in abstract data types suited for algorithmic collections.

2.12.1 Integer

Represents whole numbers: positive, negative, or zero.

```
INTEGER count = 0;
INTEGER step = -5;
```

2.12.2 Decimal

Represents numbers containing fractional values.

```
DECIMAL ratio = 0.618;
DECIMAL total = -12.50;
```

2.12.3 Character

Represents one printable ASCII character or horizontal tab enclosed in single quotation marks, following the character-literal DFA in Section 2.3.7.

```
CHARACTER grade = 'A';
CHARACTER delimiter = ',';
```

2.12.4 Boolean

Represents binary truth values defined by the literals `TRUE` and `FALSE`.

```
BOOLEAN isFound = FALSE;
BOOLEAN hasTerminated = TRUE;
```

2.12.5 Collection Types

To support data-structure modeling, LAlgo provides two native collection types:

- **String.** An immutable ordered sequence of characters enclosed in double quotation marks.

  ```
  STRING algorithmName = "Binary Search";
  ```

- **Array.** A contiguous, zero-indexed collection of homogeneous elements.

  ```
  ARRAY numbers = [2, 4, 6, 8, 10];
  INTEGER firstElement = numbers[0];
  ```

2.13 From Lexical Tokens to Statements

The lexer recognizes the words, operators, and delimiters that Chapter 3 assembles into statements. The following binary-search illustration uses the proposed syntax, including semicolons after simple statements and brace-delimited control blocks. It uses mixed output to show the matching index.

```
ARRAY numbers = [2, 4, 6, 8, 10];
INTEGER target = 8;
INTEGER low = 0;
INTEGER high = 4;
INTEGER mid = 0;
BOOLEAN found = FALSE;

WHILE (low <= high AND NOT found) {
    mid = (low + high) DIV 2;
    IF (numbers[mid] == target) {
        found = TRUE;
        OUTPUT "Found at index ", mid;
    } ELSE IF (numbers[mid] < target) {
        low = mid + 1;
    } ELSE {
        high = mid - 1;
    }
}
IF (NOT found) { OUTPUT "Not found"; }
```

3 Syntax Grammar Rules Proposal (EBNF)

This chapter defines the LAlgo syntax proposed for 12 November 2026. It covers declarations, input, output, assignment, selection, and iteration. The language uses `WHILE` for both a single loop and nested loops. The current C lexer supplies the token stream for the proposed syntax analyzer.

3.1 Grammar contract

- **Start symbol.** `Program` matches a sequence of statements followed by `EOF`. A `Block` is a brace-delimited sequence of statements and may be empty.
- **Terminals.** Quoted words and punctuation match their exact token spellings. Keywords use uppercase. The unquoted terminals below match lexer token categories.
- **Nonterminals.** Every name on the left of `=` in the numbered EBNF is a nonterminal. Each numbered production has one designated example in Section 3.7.
- **Statement boundaries.** Declarations, input, output, and assignments end with a semicolon. A conditional ends after the last block in its complete chain. A loop ends at its matching closing brace. A line break is whitespace, including between `ELSE` and `IF`.

Grammar terminal	Lexical token type
`IDENT`	`IDENT`, preserving the identifier spelling.
`INT_LIT`, `DEC_LIT`	`INTEGER`, `DECIMAL`.
`CHAR_LIT`, `STR_LIT`	`CHAR`, `STRING`, including their source quotes.
`EOF`	`EOF`, the end-of-input marker.

3.2 Reading the EBNF

A comma joins symbols in order. A vertical bar separates alternatives. Parentheses group symbols, square brackets make a group optional, and braces repeat a group zero or more times. The final semicolon ends a production definition. Quoted punctuation is part of LAlgo source.

The exception `A - B` removes any *entire token span* in `A` that also matches `B`. This gives the three assignment forms distinct right-hand sides. A bare reference or literal takes its own form. A grouped or operator-containing expression, or an array literal, takes the expression form. In `LoopBlock`, the special sequence between question marks denotes spans containing no `KW_WHILE`. Subtracting those spans leaves blocks containing a loop at any depth, including inside an `IF`. The quoted word `"WHILE"` inside a string has token type `STRING`, so it does not affect this classification.

3.3 Productions

```
(* R01 *) Program = { Statement }, EOF ;
(* R02 *) Statement = InputStmt | OutputStmt | Assignment | Declaration | Conditional
    | Loop ;
(* R03 *) Block = "{", { Statement }, "}" ;

(* R04 *) InputStmt = "INPUT", IDENT, ";" ;
(* R05 *) OutputStmt = OutputText | OutputValue | OutputMixed ;
(* R06 *) OutputText = "OUTPUT", [ "THE" ], STR_LIT, ";" ;
(* R07 *) OutputValue = "OUTPUT", [ "THE" ], Reference, ";" ;
(* R08 *) OutputMixed = "OUTPUT", [ "THE" ], STR_LIT, ",", Reference, ";" ;

(* R09 *) Assignment = AssignVariable | AssignConstant | AssignExpression ;
(* R10 *) AssignVariable = [ "SET" ], Reference, "=", Reference, ";" ;
```

```
(* R11 *) AssignConstant = [ "SET" ], Reference, "=", Literal, ";" ;
(* R12 *) AssignExpression = [ "SET" ], Reference, "=", (Expression - (Reference |
   Literal)), ";" ;

(* R13 *) Declaration = DeclareOne | DeclareList | DeclareInit ;
(* R14 *) DeclareOne = Type, IDENT, ";" ;
(* R15 *) DeclareList = Type, IDENT, ",", IDENT, { ",", IDENT }, ";" ;
(* R16 *) DeclareInit = Type, IDENT, "=", Expression, ";" ;
(* R17 *) Type = "INTEGER" | "DECIMAL" | "CHARACTER" | "BOOLEAN" | "ARRAY" | "STRING" ;

(* R18 *) Conditional = IfOnly | IfElse | IfChain ;
(* R19 *) IfHead = "IF", "(", Expression, ")", Block ;
(* R20 *) IfOnly = IfHead ;
(* R21 *) IfElse = IfHead, "ELSE", Block ;
(* R22 *) IfChain = IfHead, ElseIf, { ElseIf }, "ELSE", Block ;
(* R23 *) ElseIf = "ELSE", IfHead ;

(* R24 *) Loop = PlainWhile | NestedWhile ;
(* R25 *) WhileHead = "WHILE", "(", Expression, ")" ;
(* R26 *) PlainWhile = WhileHead, (Block - LoopBlock) ;
(* R27 *) NestedWhile = WhileHead, LoopBlock ;
(* R28 *) LoopBlock = Block - ? span contains no WHILE keyword token ? ;

(* R29 *) Expression = Conjunction, { "OR", Conjunction } ;
(* R30 *) Conjunction = Negation, { "AND", Negation } ;
(* R31 *) Negation = "NOT", Negation | Equality ;
(* R32 *) Equality = Comparison, { ( "==" | "!=" ), Comparison } ;
(* R33 *) Comparison = Sum, { ( "<" | "<=" | ">" | ">=" ), Sum } ;
(* R34 *) Sum = Product, { ( "+" | "-" ), Product } ;
(* R35 *) Product = Unary, { ( "*" | "/" | "%" | "DIV" ), Unary } ;
(* R36 *) Unary = "-", Unary | Power ;
(* R37 *) Power = Primary, [ "^", Unary ] ;
(* R38 *) Primary = Literal | Reference | "(", Expression, ")" | ArrayLiteral ;
(* R39 *) Reference = IDENT, [ "[", Expression, "]" ] ;
(* R40 *) Literal = INT_LIT | DEC_LIT | CHAR_LIT | STR_LIT | "TRUE" | "FALSE" ;
(* R41 *) ArrayLiteral = "[", [ Expression, { ",", Expression } ], "]" ;
```

The expression tiers follow the precedence table in Chapter 2. Repeated binary operators at a tier associate from the left. The recursive right operand of `Power` makes exponentiation right-associative. Thus `2 ^ 3 ^ 2` groups as `2 ^ (3 ^ 2)`. Unary minus on the left applies after exponentiation. **NOT** binds after comparison and equality and before **AND** and **OR**.

3.4 Statement coverage

Family	Designated rule examples	Forms
Input	R04	One input construct with an identifier target.
Output	R06–R08	Text only, reference value only, and text followed by a comma and a reference.
Assignment	R10–R12	Variable, literal constant, and expression as the assigned value.
Declaration	R14–R16	One name, a comma-separated list of names of one type, and one initialized name.
Condition	R20–R22	Single `IF`, `IF`–`ELSE`, and `IF` with one or more `ELSE IF` clauses and a final `ELSE`.
Iteration	R26–R27	One `WHILE` kind with a loop-free body or a body containing another `WHILE`.

Declaration with initialization starts with a type keyword. Plain assignment starts with a reference or optional `SET`. The optional `THE` appears immediately after `OUTPUT`. These two noise words have no effect on the statement's meaning.

3.5 Parsing approach and token handoff

The proposed analyzer uses **top-down recursive descent**. Its token cursor reads one terminal at a time from the lexer's in-memory token sequence. It retains line and column information, skips comment records, and reports lexical diagnostics separately. The lexer also writes `Symbol Table.txt` for lexical checking. The cursor retains `EOF` from memory.

The cursor collects terminals one at a time through the statement's terminator. It tracks delimiters so a semicolon inside a control block stays with that block. At the closing brace of an `IF` or `ELSE IF` body, one terminal of lookahead determines whether `ELSE` extends the statement. Once the statement boundary is reached, recursive descent validates the collected terminals and the statements within any nested blocks.

The first terminal selects the statement family. A declaration's separator distinguishes a single name, list, or initializer. After `OUTPUT`, a string followed by a comma selects the mixed form. An assignment is classified from the complete shape of its right-hand expression. A conditional chain with an `ELSE IF` requires a final `ELSE`. For a `WHILE`, the parsed body determines which loop rule applies. The analyzer shares common prefixes and records the corresponding named production in the tree.

3.6 Syntax errors and panic recovery

On a syntax error, report the unexpected token and the expected token or construct, with its line and column. For example, the proposed format is `Syntax error on line 3, column 1: expected ';' after assignment.`

Panic recovery discards the damaged part of the statement. The recovery scan tracks grouping and block delimiters and looks for a semicolon, the current block's closing brace, or EOF at a suitable nesting level. It may also resume at an unambiguous next statement starter at the same level, such as a type or I/O keyword after a missing semicolon. The parser consumes a synchronizing semicolon but leaves a closing brace for the enclosing block. Each recovery step consumes input or returns control to an enclosing rule. The analyzer continues checking later statements and reports failure for the complete program if any error was recorded.

Semantic checking is scheduled separately. It will use the syntax tree to check types and declarations and maintain semantic symbol-table entries. The grammar examples demonstrate syntactic form, so an identifier in a fragment can stand for an already-declared name.

3.7 One example, two derivations, and one parse tree per rule

Each numbered rule has one designated source example. A fragment starts at the rule it illustrates. Each derivation starts with that rule's nonterminal. Every line beginning with `=>` shows the full result after expanding one nonterminal. The leftmost derivation expands the leftmost remaining nonterminal, and the rightmost derivation expands the rightmost one. Indented continuation lines belong to the preceding step.

One bracketed parse tree shows the structure shared by the two derivations. In `Name[ ... ]`, `Name` is the parent and the entries in brackets are its children in source order. An entry such as `IDENT:age` gives a terminal category and its lexeme. An absent optional or empty repetition contributes no child.

R01 `Program`

```
INTEGER count = 1; OUTPUT count;
```

Leftmost derivation

```
Program
=> Statement Statement EOF
=> Declaration Statement EOF
=> DeclareInit Statement EOF
=> Type IDENT:count '=' Expression ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Expression ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Conjunction ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Negation ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Equality ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Comparison ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Sum ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Product ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Unary ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Power ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Primary ';' Statement EOF
=> 'INTEGER' IDENT:count '=' Literal ';' Statement EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' Statement EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' OutputStmt EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' OutputValue EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' 'OUTPUT' Reference ';' EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' 'OUTPUT' IDENT:count ';' EOF
```

Rightmost derivation

```
Program
=> Statement Statement EOF
=> Statement OutputStmt EOF
=> Statement OutputValue EOF
=> Statement 'OUTPUT' Reference ';' EOF
=> Statement 'OUTPUT' IDENT:count ';' EOF
=> Declaration 'OUTPUT' IDENT:count ';' EOF
=> DeclareInit 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Expression ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Conjunction ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Negation ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Equality ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Comparison ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Sum ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Product ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Unary ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Power ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Primary ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' Literal ';' 'OUTPUT' IDENT:count ';' EOF
=> Type IDENT:count '=' INT_LIT:1 ';' 'OUTPUT' IDENT:count ';' EOF
=> 'INTEGER' IDENT:count '=' INT_LIT:1 ';' 'OUTPUT' IDENT:count ';' EOF
```

Parse tree

```
Program[ Statement[ Declaration[ DeclareInit[ Type[ 'INTEGER' ] IDENT:count '=' Expression[
   Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Literal[
   INT_LIT:1 ] ] ] ] ] ] ] ] ] ] ] ';' ] ] ] Statement[ OutputStmt[ OutputValue[ 'OUTPUT' Reference[
   IDENT:count ] ';' ] ] ] EOF ]
```

R02 Statement

```
INPUT limit;
```

Leftmost derivation

```
Statement
=> InputStmt
=> 'INPUT' IDENT:limit ';'
```

Rightmost derivation

```
Statement
=> InputStmt
=> 'INPUT' IDENT:limit ';'
```

Parse tree

```
Statement[ InputStmt[ 'INPUT' IDENT:limit ';' ] ]
```

R03 Block

```
{ OUTPUT "ready"; }
```

Leftmost derivation

```
Block
=> '{' Statement '}'
=> '{' OutputStmt '}'
=> '{' OutputText '}'
=> '{' 'OUTPUT' STR_LIT:"ready" ';' '}'
```

Rightmost derivation

```
Block
=> '{' Statement '}'
=> '{' OutputStmt '}'
=> '{' OutputText '}'
=> '{' 'OUTPUT' STR_LIT:"ready" ';' '}'
```

Parse tree

```
Block[ '{' Statement[ OutputStmt[ OutputText[ 'OUTPUT' STR_LIT:"ready" ';' ] ] ] '}' ]
```

R04 InputStmt

```
INPUT age;
```

Leftmost derivation

```
InputStmt
=> 'INPUT' IDENT:age ';'
```

Rightmost derivation

```
InputStmt
=> 'INPUT' IDENT:age ';'
```

Parse tree

```
InputStmt[ 'INPUT' IDENT:age ';' ]
```

R05 OutputStmt

```
OUTPUT THE total;
```

Leftmost derivation

```
OutputStmt
=> OutputValue
=> 'OUTPUT' 'THE' Reference ';'
=> 'OUTPUT' 'THE' IDENT:total ';'
```

Rightmost derivation

```
OutputStmt
=> OutputValue
=> 'OUTPUT' 'THE' Reference ';'
=> 'OUTPUT' 'THE' IDENT:total ';'
```

Parse tree

```
OutputStmt[ OutputValue[ 'OUTPUT' 'THE' Reference[ IDENT:total ] ';' ] ]
```

R06 OutputText

```
OUTPUT "Hello world";
```

Leftmost derivation

```
OutputText
=> 'OUTPUT' STR_LIT:"Hello world" ';'
```

Rightmost derivation

```
OutputText
=> 'OUTPUT' STR_LIT:"Hello world" ';'
```

Parse tree

```
OutputText[ 'OUTPUT' STR_LIT:"Hello world" ';' ]
```

R07 OutputValue

```
OUTPUT score;
```

Leftmost derivation

```
OutputValue
=> 'OUTPUT' Reference ';'
=> 'OUTPUT' IDENT:score ';'
```

Rightmost derivation

```
OutputValue
=> 'OUTPUT' Reference ';'
=> 'OUTPUT' IDENT:score ';'
```

Parse tree

```
OutputValue[ 'OUTPUT' Reference[ IDENT:score ] ';' ]
```

R08 OutputMixed

```
OUTPUT "Age: ", age;
```

Leftmost derivation

```
OutputMixed
=> 'OUTPUT' STR_LIT:"Age: " ',' Reference ';'
=> 'OUTPUT' STR_LIT:"Age: " ',' IDENT:age ';'
```

Rightmost derivation

```
OutputMixed
=> 'OUTPUT' STR_LIT:"Age: " ',' Reference ';'
=> 'OUTPUT' STR_LIT:"Age: " ',' IDENT:age ';'
```

Parse tree

```
OutputMixed[ 'OUTPUT' STR_LIT:"Age: " ',' Reference[ IDENT:age ] ';' ]
```

R09 Assignment

```
SET total = count;
```

Leftmost derivation

```
Assignment
=> AssignVariable
=> 'SET' Reference '=' Reference ';'
=> 'SET' IDENT:total '=' Reference ';'
=> 'SET' IDENT:total '=' IDENT:count ';'
```

Rightmost derivation

```
Assignment
=> AssignVariable
=> 'SET' Reference '=' Reference ';'
=> 'SET' Reference '=' IDENT:count ';'
=> 'SET' IDENT:total '=' IDENT:count ';'
```

Parse tree

```
Assignment[ AssignVariable[ 'SET' Reference[ IDENT:total ] '=' Reference[ IDENT:count ] ';' ] ]
```

R10 AssignVariable

```
a = b;
```

Leftmost derivation

```
AssignVariable
=> Reference '=' Reference ';'
=> IDENT:a '=' Reference ';'
=> IDENT:a '=' IDENT:b ';'
```

Rightmost derivation

```
AssignVariable
=> Reference '=' Reference ';'
=> Reference '=' IDENT:b ';'
=> IDENT:a '=' IDENT:b ';'
```

Parse tree

```
AssignVariable[ Reference[ IDENT:a ] '=' Reference[ IDENT:b ] ';' ]
```

R11 AssignConstant

```
a = 1;
```

Leftmost derivation

```
AssignConstant
=> Reference '=' Literal ';'
=> IDENT:a '=' Literal ';'
=> IDENT:a '=' INT_LIT:1 ';'
```

Rightmost derivation

```
AssignConstant
=> Reference '=' Literal ';'
=> Reference '=' INT_LIT:1 ';'
=> IDENT:a '=' INT_LIT:1 ';'
```

Parse tree

```
AssignConstant[ Reference[ IDENT:a ] '=' Literal[ INT_LIT:1 ] ';' ]
```

R12 AssignExpression

```
a = 3 + 2 * 5;
```

Leftmost derivation

```
AssignExpression
=> Reference '=' Expression ';'
=> IDENT:a '=' Expression ';'
=> IDENT:a '=' Conjunction ';'
=> IDENT:a '=' Negation ';'
=> IDENT:a '=' Equality ';'
=> IDENT:a '=' Comparison ';'
=> IDENT:a '=' Sum ';'
=> IDENT:a '=' Product '+' Product ';'
=> IDENT:a '=' Unary '+' Product ';'
=> IDENT:a '=' Power '+' Product ';'
=> IDENT:a '=' Primary '+' Product ';'
=> IDENT:a '=' Literal '+' Product ';'
=> IDENT:a '=' INT_LIT:3 '+' Product ';'
=> IDENT:a '=' INT_LIT:3 '+' Unary '*' Unary ';'
=> IDENT:a '=' INT_LIT:3 '+' Power '*' Unary ';'
=> IDENT:a '=' INT_LIT:3 '+' Primary '*' Unary ';'
=> IDENT:a '=' INT_LIT:3 '+' Literal '*' Unary ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' Unary ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' Power ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' Primary ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' Literal ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' INT_LIT:5 ';'
```

Rightmost derivation

```
AssignExpression
=> Reference '=' Expression ';'
=> Reference '=' Conjunction ';'
=> Reference '=' Negation ';'
=> Reference '=' Equality ';'
=> Reference '=' Comparison ';'
=> Reference '=' Sum ';'
=> Reference '=' Product '+' Product ';'
=> Reference '=' Product '+' Unary '*' Unary ';'
=> Reference '=' Product '+' Unary '*' Power ';'
=> Reference '=' Product '+' Unary '*' Primary ';'
=> Reference '=' Product '+' Unary '*' Literal ';'
=> Reference '=' Product '+' Unary '*' INT_LIT:5 ';'
=> Reference '=' Product '+' Power '*' INT_LIT:5 ';'
=> Reference '=' Product '+' Primary '*' INT_LIT:5 ';'
=> Reference '=' Product '+' Literal '*' INT_LIT:5 ';'
=> Reference '=' Product '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> Reference '=' Unary '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> Reference '=' Power '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> Reference '=' Primary '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> Reference '=' Literal '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> Reference '=' INT_LIT:3 '+' INT_LIT:2 '*' INT_LIT:5 ';'
=> IDENT:a '=' INT_LIT:3 '+' INT_LIT:2 '*' INT_LIT:5 ';'
```

Parse tree

```
AssignExpression[ Reference[ IDENT:a ] '=' Expression[ Conjunction[ Negation[ Equality[ Comparison[
   Sum[ Product[ Unary[ Power[ Primary[ Literal[ INT_LIT:3 ] ] ] ] ] '+' Product[ Unary[ Power[
   Primary[ Literal[ INT_LIT:2 ] ] ] ] '*' Unary[ Power[ Primary[ Literal[ INT_LIT:5 ] ] ] ] ] ] ] ]
   ] ] ] ';' ]
```

R13 Declaration

```
DECIMAL rate;
```

Leftmost derivation

```
Declaration
=> DeclareOne
=> Type IDENT:rate ';'
=> 'DECIMAL' IDENT:rate ';'
```

Rightmost derivation

```
Declaration
=> DeclareOne
=> Type IDENT:rate ';'
=> 'DECIMAL' IDENT:rate ';'
```

Parse tree

```
Declaration[ DeclareOne[ Type[ 'DECIMAL' ] IDENT:rate ';' ] ]
```

R14 DeclareOne

```
INTEGER count;
```

Leftmost derivation

```
DeclareOne
=> Type IDENT:count ';'
=> 'INTEGER' IDENT:count ';'
```

Rightmost derivation

```
DeclareOne
=> Type IDENT:count ';'
=> 'INTEGER' IDENT:count ';'
```

Parse tree

```
DeclareOne[ Type[ 'INTEGER' ] IDENT:count ';' ]
```

R15 DeclareList

```
INTEGER low, mid, high;
```

Leftmost derivation

```
DeclareList
=> Type IDENT:low ',' IDENT:mid ',' IDENT:high ';'
=> 'INTEGER' IDENT:low ',' IDENT:mid ',' IDENT:high ';'
```

Rightmost derivation

```
DeclareList
=> Type IDENT:low ',' IDENT:mid ',' IDENT:high ';'
=> 'INTEGER' IDENT:low ',' IDENT:mid ',' IDENT:high ';'
```

Parse tree

```
DeclareList[ Type[ 'INTEGER' ] IDENT:low ',' IDENT:mid ',' IDENT:high ';' ]
```

R16 DeclareInit

```
STRING name = "Ada";
```

Leftmost derivation

```
DeclareInit
=> Type IDENT:name '=' Expression ';'
=> 'STRING' IDENT:name '=' Expression ';'
=> 'STRING' IDENT:name '=' Conjunction ';'
=> 'STRING' IDENT:name '=' Negation ';'
=> 'STRING' IDENT:name '=' Equality ';'
=> 'STRING' IDENT:name '=' Comparison ';'
```

```
=> 'STRING' IDENT:name '=' Sum ';'
=> 'STRING' IDENT:name '=' Product ';'
=> 'STRING' IDENT:name '=' Unary ';'
=> 'STRING' IDENT:name '=' Power ';'
=> 'STRING' IDENT:name '=' Primary ';'
=> 'STRING' IDENT:name '=' Literal ';'
=> 'STRING' IDENT:name '=' STR_LIT:"Ada" ';'
```

Rightmost derivation

```
DeclareInit
=> Type IDENT:name '=' Expression ';'
=> Type IDENT:name '=' Conjunction ';'
=> Type IDENT:name '=' Negation ';'
=> Type IDENT:name '=' Equality ';'
=> Type IDENT:name '=' Comparison ';'
=> Type IDENT:name '=' Sum ';'
=> Type IDENT:name '=' Product ';'
=> Type IDENT:name '=' Unary ';'
=> Type IDENT:name '=' Power ';'
=> Type IDENT:name '=' Primary ';'
=> Type IDENT:name '=' Literal ';'
=> Type IDENT:name '=' STR_LIT:"Ada" ';'
=> 'STRING' IDENT:name '=' STR_LIT:"Ada" ';'
```

Parse tree

```
DeclareInit[ Type[ 'STRING' ] IDENT:name '=' Expression[ Conjunction[ Negation[ Equality[
   Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Literal[ STR_LIT:"Ada" ] ] ] ] ] ] ] ] ] ] ] ';'
   ]
```

R17 Type

```
BOOLEAN
```

Leftmost derivation

```
Type
=> 'BOOLEAN'
```

Rightmost derivation

```
Type
=> 'BOOLEAN'
```

Parse tree

```
Type[ 'BOOLEAN' ]
```

R18 Conditional

```
IF (ready) { OUTPUT flag; }
```

Leftmost derivation

```
Conditional
=> IfOnly
=> IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Conjunction ')' Block
=> 'IF' '(' Negation ')' Block
=> 'IF' '(' Equality ')' Block
=> 'IF' '(' Comparison ')' Block
=> 'IF' '(' Sum ')' Block
=> 'IF' '(' Product ')' Block
=> 'IF' '(' Unary ')' Block
=> 'IF' '(' Power ')' Block
=> 'IF' '(' Primary ')' Block
=> 'IF' '(' Reference ')' Block
=> 'IF' '(' IDENT:ready ')' Block
=> 'IF' '(' IDENT:ready ')' '{' Statement '}'
=> 'IF' '(' IDENT:ready ')' '{' OutputStmt '}'
=> 'IF' '(' IDENT:ready ')' '{' OutputValue '}'
=> 'IF' '(' IDENT:ready ')' '{' 'OUTPUT' Reference ';' '}'
=> 'IF' '(' IDENT:ready ')' '{' 'OUTPUT' IDENT:flag ';' '}'
```

Rightmost derivation

```
Conditional
=> IfOnly
=> IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Expression ')' '{' Statement '}'
=> 'IF' '(' Expression ')' '{' OutputStmt '}'
=> 'IF' '(' Expression ')' '{' OutputValue '}'
=> 'IF' '(' Expression ')' '{' 'OUTPUT' Reference ';' '}'
=> 'IF' '(' Expression ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Conjunction ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Negation ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Equality ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Comparison ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Sum ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Product ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Unary ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Power ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Primary ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' Reference ')' '{' 'OUTPUT' IDENT:flag ';' '}'
=> 'IF' '(' IDENT:ready ')' '{' 'OUTPUT' IDENT:flag ';' '}'
```

Parse tree

```
Conditional[ IfOnly[ IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:ready ] ] ] ] ] ] ] ] ] ] ] ')' Block[ '{'
   Statement[ OutputStmt[ OutputValue[ 'OUTPUT' Reference[ IDENT:flag ] ';' ] ] ] '}' ] ] ] ]
```

R19 IfHead

```
IF (TRUE) {}
```

Leftmost derivation

```
IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Conjunction ')' Block
=> 'IF' '(' Negation ')' Block
=> 'IF' '(' Equality ')' Block
=> 'IF' '(' Comparison ')' Block
=> 'IF' '(' Sum ')' Block
=> 'IF' '(' Product ')' Block
=> 'IF' '(' Unary ')' Block
=> 'IF' '(' Power ')' Block
=> 'IF' '(' Primary ')' Block
=> 'IF' '(' Literal ')' Block
=> 'IF' '(' 'TRUE' ')' Block
=> 'IF' '(' 'TRUE' ')' '{' '}'
```

Rightmost derivation

```
IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Expression ')' '{' '}'
=> 'IF' '(' Conjunction ')' '{' '}'
=> 'IF' '(' Negation ')' '{' '}'
=> 'IF' '(' Equality ')' '{' '}'
=> 'IF' '(' Comparison ')' '{' '}'
=> 'IF' '(' Sum ')' '{' '}'
=> 'IF' '(' Product ')' '{' '}'
=> 'IF' '(' Unary ')' '{' '}'
=> 'IF' '(' Power ')' '{' '}'
=> 'IF' '(' Primary ')' '{' '}'
=> 'IF' '(' Literal ')' '{' '}'
=> 'IF' '(' 'TRUE' ')' '{' '}'
```

Parse tree

```
IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[
   Power[ Primary[ Literal[ 'TRUE' ] ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' '}' ] ]
```

R20 IfOnly

```
IF (found) { OUTPUT "Found"; }
```

Leftmost derivation

```
IfOnly
=> IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Conjunction ')' Block
=> 'IF' '(' Negation ')' Block
=> 'IF' '(' Equality ')' Block
=> 'IF' '(' Comparison ')' Block
=> 'IF' '(' Sum ')' Block
=> 'IF' '(' Product ')' Block
=> 'IF' '(' Unary ')' Block
=> 'IF' '(' Power ')' Block
=> 'IF' '(' Primary ')' Block
=> 'IF' '(' Reference ')' Block
=> 'IF' '(' IDENT:found ')' Block
=> 'IF' '(' IDENT:found ')' '{' Statement '}'
=> 'IF' '(' IDENT:found ')' '{' OutputStmt '}'
=> 'IF' '(' IDENT:found ')' '{' OutputText '}'
=> 'IF' '(' IDENT:found ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
```

Rightmost derivation

```
IfOnly
=> IfHead
=> 'IF' '(' Expression ')' Block
=> 'IF' '(' Expression ')' '{' Statement '}'
=> 'IF' '(' Expression ')' '{' OutputStmt '}'
=> 'IF' '(' Expression ')' '{' OutputText '}'
=> 'IF' '(' Expression ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Conjunction ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Negation ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Equality ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Comparison ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Sum ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Product ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Unary ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Power ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Primary ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' Reference ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
=> 'IF' '(' IDENT:found ')' '{' 'OUTPUT' STR_LIT:"Found" ';' '}'
```

Parse tree

```
IfOnly[ IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[
   Unary[ Power[ Primary[ Reference[ IDENT:found ] ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' Statement[
   OutputStmt[ OutputText[ 'OUTPUT' STR_LIT:"Found" ';' ] ] ] '}' ] ] ]
```

R21 IfElse

```
IF (done) {} ELSE { OUTPUT "Wait"; }
```

Leftmost derivation

```
IfElse
=> IfHead 'ELSE' Block
=> 'IF' '(' Expression ')' Block 'ELSE' Block
=> 'IF' '(' Conjunction ')' Block 'ELSE' Block
=> 'IF' '(' Negation ')' Block 'ELSE' Block
=> 'IF' '(' Equality ')' Block 'ELSE' Block
=> 'IF' '(' Comparison ')' Block 'ELSE' Block
=> 'IF' '(' Sum ')' Block 'ELSE' Block
=> 'IF' '(' Product ')' Block 'ELSE' Block
=> 'IF' '(' Unary ')' Block 'ELSE' Block
=> 'IF' '(' Power ')' Block 'ELSE' Block
=> 'IF' '(' Primary ')' Block 'ELSE' Block
=> 'IF' '(' Reference ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:done ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' Block
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' '{' Statement '}'
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' '{' OutputStmt '}'
```

```
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' '{' OutputText '}'
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
```

Rightmost derivation

```
IfElse
=> IfHead 'ELSE' Block
=> IfHead 'ELSE' '{' Statement '}'
=> IfHead 'ELSE' '{' OutputStmt '}'
=> IfHead 'ELSE' '{' OutputText '}'
=> IfHead 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Expression ')' Block 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Expression ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Conjunction ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Negation ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Equality ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Comparison ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Sum ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Product ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Unary ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Power ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Primary ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' Reference ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
=> 'IF' '(' IDENT:done ')' '{' '}' 'ELSE' '{' 'OUTPUT' STR_LIT:"Wait" ';' '}'
```

Parse tree

```
IfElse[ IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[
   Unary[ Power[ Primary[ Reference[ IDENT:done ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' '}' ] ] 'ELSE'
   Block[ '{' Statement[ OutputStmt[ OutputText[ 'OUTPUT' STR_LIT:"Wait" ';' ] ] ] '}' ] ]
```

R22 IfChain

```
IF (first) {} ELSE IF (second) {} ELSE IF (third) {} ELSE {}
```

Leftmost derivation

```
IfChain
=> IfHead ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Expression ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Conjunction ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Negation ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Equality ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Comparison ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Sum ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Product ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Unary ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Power ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Primary ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' Reference ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' Block ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' ElseIf ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' IfHead ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Expression ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Conjunction ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Negation ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Equality ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Comparison ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Sum ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Product ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Unary ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Power ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Primary ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' Reference ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' Block ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' ElseIf 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' IfHead 'ELSE'
   Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   Expression ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   Conjunction ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   Negation ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
```

```
   Equality ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   Comparison ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' Sum ')'
   Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' Product
   ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' Unary
   ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' Power
   ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' Primary
   ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   Reference ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' Block 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' Block
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
```

Rightmost derivation

```
IfChain
=> IfHead ElseIf ElseIf 'ELSE' Block
=> IfHead ElseIf ElseIf 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' IfHead 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Expression ')' Block 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Expression ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Conjunction ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Negation ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Equality ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Comparison ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Sum ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Product ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Unary ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Power ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Primary ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' Reference ')' '{' '}' 'ELSE' '{' '}'
=> IfHead ElseIf 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' IfHead 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Expression ')' Block 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Expression ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Conjunction ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Negation ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Equality ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Comparison ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' Sum ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Product ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Unary ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Power ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Primary ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> IfHead 'ELSE' 'IF' '(' Reference ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE' '{'
   '}'
=> IfHead 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')' '{' '}' 'ELSE'
   '{' '}'
=> 'IF' '(' Expression ')' Block 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Expression ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Conjunction ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Negation ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Equality ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Comparison ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Sum ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third ')'
```

```
   '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Product ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third
   ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Unary ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third
   ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Power ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third
   ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Primary ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '(' IDENT:third
   ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' Reference ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
=> 'IF' '(' IDENT:first ')' '{' '}' 'ELSE' 'IF' '(' IDENT:second ')' '{' '}' 'ELSE' 'IF' '('
   IDENT:third ')' '{' '}' 'ELSE' '{' '}'
```

Parse tree

```
IfChain[ IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[
   Unary[ Power[ Primary[ Reference[ IDENT:first ] ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' '}' ] ]
   ElseIf[ 'ELSE' IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:second ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' '}'
   ] ] ] ElseIf[ 'ELSE' IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[
   Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:third ] ] ] ] ] ] ] ] ] ] ')' Block[ '{'
   '}' ] ] ] 'ELSE' Block[ '{' '}' ] ]
```

R23 ElseIf

```
ELSE IF (more) {}
```

Leftmost derivation

```
ElseIf
=> 'ELSE' IfHead
=> 'ELSE' 'IF' '(' Expression ')' Block
=> 'ELSE' 'IF' '(' Conjunction ')' Block
=> 'ELSE' 'IF' '(' Negation ')' Block
=> 'ELSE' 'IF' '(' Equality ')' Block
=> 'ELSE' 'IF' '(' Comparison ')' Block
=> 'ELSE' 'IF' '(' Sum ')' Block
=> 'ELSE' 'IF' '(' Product ')' Block
=> 'ELSE' 'IF' '(' Unary ')' Block
=> 'ELSE' 'IF' '(' Power ')' Block
=> 'ELSE' 'IF' '(' Primary ')' Block
=> 'ELSE' 'IF' '(' Reference ')' Block
=> 'ELSE' 'IF' '(' IDENT:more ')' Block
=> 'ELSE' 'IF' '(' IDENT:more ')' '{' '}'
```

Rightmost derivation

```
ElseIf
=> 'ELSE' IfHead
=> 'ELSE' 'IF' '(' Expression ')' Block
=> 'ELSE' 'IF' '(' Expression ')' '{' '}'
=> 'ELSE' 'IF' '(' Conjunction ')' '{' '}'
=> 'ELSE' 'IF' '(' Negation ')' '{' '}'
=> 'ELSE' 'IF' '(' Equality ')' '{' '}'
=> 'ELSE' 'IF' '(' Comparison ')' '{' '}'
=> 'ELSE' 'IF' '(' Sum ')' '{' '}'
=> 'ELSE' 'IF' '(' Product ')' '{' '}'
=> 'ELSE' 'IF' '(' Unary ')' '{' '}'
=> 'ELSE' 'IF' '(' Power ')' '{' '}'
=> 'ELSE' 'IF' '(' Primary ')' '{' '}'
=> 'ELSE' 'IF' '(' Reference ')' '{' '}'
=> 'ELSE' 'IF' '(' IDENT:more ')' '{' '}'
```

Parse tree

```
ElseIf[ 'ELSE' IfHead[ 'IF' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:more ] ] ] ] ] ] ] ] ] ] ')' Block[ '{' '}' ]
   ] ]
```

R24 Loop

```
WHILE (ready) {}
```

Leftmost derivation

```
Loop
=> PlainWhile
=> WhileHead Block
=> 'WHILE' '(' Expression ')' Block
=> 'WHILE' '(' Conjunction ')' Block
=> 'WHILE' '(' Negation ')' Block
=> 'WHILE' '(' Equality ')' Block
=> 'WHILE' '(' Comparison ')' Block
=> 'WHILE' '(' Sum ')' Block
=> 'WHILE' '(' Product ')' Block
=> 'WHILE' '(' Unary ')' Block
=> 'WHILE' '(' Power ')' Block
=> 'WHILE' '(' Primary ')' Block
=> 'WHILE' '(' Reference ')' Block
=> 'WHILE' '(' IDENT:ready ')' Block
=> 'WHILE' '(' IDENT:ready ')' '{' '}'
```

Rightmost derivation

```
Loop
=> PlainWhile
=> WhileHead Block
=> WhileHead '{' '}'
=> 'WHILE' '(' Expression ')' '{' '}'
=> 'WHILE' '(' Conjunction ')' '{' '}'
=> 'WHILE' '(' Negation ')' '{' '}'
=> 'WHILE' '(' Equality ')' '{' '}'
=> 'WHILE' '(' Comparison ')' '{' '}'
=> 'WHILE' '(' Sum ')' '{' '}'
=> 'WHILE' '(' Product ')' '{' '}'
=> 'WHILE' '(' Unary ')' '{' '}'
=> 'WHILE' '(' Power ')' '{' '}'
=> 'WHILE' '(' Primary ')' '{' '}'
=> 'WHILE' '(' Reference ')' '{' '}'
=> 'WHILE' '(' IDENT:ready ')' '{' '}'
```

Parse tree

```
Loop[ PlainWhile[ WhileHead[ 'WHILE' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[
   Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:ready ] ] ] ] ] ] ] ] ] ] ] ')' ] Block[
   '{' '}' ] ] ]
```

R25 WhileHead

```
WHILE (running)
```

Leftmost derivation

```
WhileHead
=> 'WHILE' '(' Expression ')'
=> 'WHILE' '(' Conjunction ')'
=> 'WHILE' '(' Negation ')'
=> 'WHILE' '(' Equality ')'
=> 'WHILE' '(' Comparison ')'
=> 'WHILE' '(' Sum ')'
=> 'WHILE' '(' Product ')'
=> 'WHILE' '(' Unary ')'
=> 'WHILE' '(' Power ')'
=> 'WHILE' '(' Primary ')'
=> 'WHILE' '(' Reference ')'
=> 'WHILE' '(' IDENT:running ')'
```

Rightmost derivation

```
WhileHead
=> 'WHILE' '(' Expression ')'
=> 'WHILE' '(' Conjunction ')'
=> 'WHILE' '(' Negation ')'
=> 'WHILE' '(' Equality ')'
=> 'WHILE' '(' Comparison ')'
```

```
=> 'WHILE' '(' Sum ')'
=> 'WHILE' '(' Product ')'
=> 'WHILE' '(' Unary ')'
=> 'WHILE' '(' Power ')'
=> 'WHILE' '(' Primary ')'
=> 'WHILE' '(' Reference ')'
=> 'WHILE' '(' IDENT:running ')'
```

Parse tree

```
WhileHead[ 'WHILE' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[
   Power[ Primary[ Reference[ IDENT:running ] ] ] ] ] ] ] ] ] ] ] ')' ]
```

R26 PlainWhile

```
WHILE (active) { OUTPUT active; }
```

Leftmost derivation

```
PlainWhile
=> WhileHead Block
=> 'WHILE' '(' Expression ')' Block
=> 'WHILE' '(' Conjunction ')' Block
=> 'WHILE' '(' Negation ')' Block
=> 'WHILE' '(' Equality ')' Block
=> 'WHILE' '(' Comparison ')' Block
=> 'WHILE' '(' Sum ')' Block
=> 'WHILE' '(' Product ')' Block
=> 'WHILE' '(' Unary ')' Block
=> 'WHILE' '(' Power ')' Block
=> 'WHILE' '(' Primary ')' Block
=> 'WHILE' '(' Reference ')' Block
=> 'WHILE' '(' IDENT:active ')' Block
=> 'WHILE' '(' IDENT:active ')' '{' Statement '}'
=> 'WHILE' '(' IDENT:active ')' '{' OutputStmt '}'
=> 'WHILE' '(' IDENT:active ')' '{' OutputValue '}'
=> 'WHILE' '(' IDENT:active ')' '{' 'OUTPUT' Reference ';' '}'
=> 'WHILE' '(' IDENT:active ')' '{' 'OUTPUT' IDENT:active ';' '}'
```

Rightmost derivation

```
PlainWhile
=> WhileHead Block
=> WhileHead '{' Statement '}'
=> WhileHead '{' OutputStmt '}'
=> WhileHead '{' OutputValue '}'
=> WhileHead '{' 'OUTPUT' Reference ';' '}'
=> WhileHead '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Expression ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Conjunction ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Negation ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Equality ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Comparison ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Sum ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Product ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Unary ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Power ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Primary ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' Reference ')' '{' 'OUTPUT' IDENT:active ';' '}'
=> 'WHILE' '(' IDENT:active ')' '{' 'OUTPUT' IDENT:active ';' '}'
```

Parse tree

```
PlainWhile[ WhileHead[ 'WHILE' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:active ] ] ] ] ] ] ] ] ] ] ] ')' ] Block[ '{'
   Statement[ OutputStmt[ OutputValue[ 'OUTPUT' Reference[ IDENT:active ] ';' ] ] ] '}' ] ]
```

R27 NestedWhile

```
WHILE (outer) { WHILE (inner) {} }
```

Leftmost derivation

```
NestedWhile
=> WhileHead LoopBlock
=> 'WHILE' '(' Expression ')' LoopBlock
=> 'WHILE' '(' Conjunction ')' LoopBlock
=> 'WHILE' '(' Negation ')' LoopBlock
=> 'WHILE' '(' Equality ')' LoopBlock
=> 'WHILE' '(' Comparison ')' LoopBlock
=> 'WHILE' '(' Sum ')' LoopBlock
=> 'WHILE' '(' Product ')' LoopBlock
=> 'WHILE' '(' Unary ')' LoopBlock
=> 'WHILE' '(' Power ')' LoopBlock
=> 'WHILE' '(' Primary ')' LoopBlock
=> 'WHILE' '(' Reference ')' LoopBlock
=> 'WHILE' '(' IDENT:outer ')' LoopBlock
=> 'WHILE' '(' IDENT:outer ')' Block
=> 'WHILE' '(' IDENT:outer ')' '{' Statement '}'
=> 'WHILE' '(' IDENT:outer ')' '{' Loop '}'
=> 'WHILE' '(' IDENT:outer ')' '{' PlainWhile '}'
=> 'WHILE' '(' IDENT:outer ')' '{' WhileHead Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Expression ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Conjunction ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Negation ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Equality ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Comparison ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Sum ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Product ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Unary ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Power ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Primary ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' Reference ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' IDENT:inner ')' Block '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
```

Rightmost derivation

```
NestedWhile
=> WhileHead LoopBlock
=> WhileHead Block
=> WhileHead '{' Statement '}'
=> WhileHead '{' Loop '}'
=> WhileHead '{' PlainWhile '}'
=> WhileHead '{' WhileHead Block '}'
=> WhileHead '{' WhileHead '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Expression ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Conjunction ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Negation ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Equality ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Comparison ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Sum ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Product ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Unary ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Power ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Primary ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' Reference ')' '{' '}' '}'
=> WhileHead '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Expression ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Conjunction ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Negation ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Equality ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Comparison ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Sum ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Product ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Unary ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Power ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Primary ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' Reference ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
=> 'WHILE' '(' IDENT:outer ')' '{' 'WHILE' '(' IDENT:inner ')' '{' '}' '}'
```

Parse tree

```
NestedWhile[ WhileHead[ 'WHILE' '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:outer ] ] ] ] ] ] ] ] ] ] ] ] ')' ] LoopBlock[
   Block[ '{' Statement[ Loop[ PlainWhile[ WhileHead[ 'WHILE' '(' Expression[ Conjunction[ Negation[
   Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:inner ] ] ] ] ] ] ] ]
   ] ] ] ')' ] Block[ '{' '}' ] ] ] ] '}' ] ] ]
```

R28 LoopBlock

```
{ WHILE (FALSE) {} }
```

Leftmost derivation

```
LoopBlock
=> Block
=> '{' Statement '}'
=> '{' Loop '}'
=> '{' PlainWhile '}'
=> '{' WhileHead Block '}'
=> '{' 'WHILE' '(' Expression ')' Block '}'
=> '{' 'WHILE' '(' Conjunction ')' Block '}'
=> '{' 'WHILE' '(' Negation ')' Block '}'
=> '{' 'WHILE' '(' Equality ')' Block '}'
=> '{' 'WHILE' '(' Comparison ')' Block '}'
=> '{' 'WHILE' '(' Sum ')' Block '}'
=> '{' 'WHILE' '(' Product ')' Block '}'
=> '{' 'WHILE' '(' Unary ')' Block '}'
=> '{' 'WHILE' '(' Power ')' Block '}'
=> '{' 'WHILE' '(' Primary ')' Block '}'
=> '{' 'WHILE' '(' Literal ')' Block '}'
=> '{' 'WHILE' '(' 'FALSE' ')' Block '}'
=> '{' 'WHILE' '(' 'FALSE' ')' '{' '}' '}'
```

Rightmost derivation

```
LoopBlock
=> Block
=> '{' Statement '}'
=> '{' Loop '}'
=> '{' PlainWhile '}'
=> '{' WhileHead Block '}'
=> '{' WhileHead '{' '}' '}'
=> '{' 'WHILE' '(' Expression ')' '{' '}' '}'
=> '{' 'WHILE' '(' Conjunction ')' '{' '}' '}'
=> '{' 'WHILE' '(' Negation ')' '{' '}' '}'
=> '{' 'WHILE' '(' Equality ')' '{' '}' '}'
=> '{' 'WHILE' '(' Comparison ')' '{' '}' '}'
=> '{' 'WHILE' '(' Sum ')' '{' '}' '}'
=> '{' 'WHILE' '(' Product ')' '{' '}' '}'
=> '{' 'WHILE' '(' Unary ')' '{' '}' '}'
=> '{' 'WHILE' '(' Power ')' '{' '}' '}'
=> '{' 'WHILE' '(' Primary ')' '{' '}' '}'
=> '{' 'WHILE' '(' Literal ')' '{' '}' '}'
=> '{' 'WHILE' '(' 'FALSE' ')' '{' '}' '}'
```

Parse tree

```
LoopBlock[ Block[ '{' Statement[ Loop[ PlainWhile[ WhileHead[ 'WHILE' '(' Expression[ Conjunction[
   Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Literal[ 'FALSE' ] ] ] ] ] ]
   ] ] ] ] ] ')' ] Block[ '{' '}' ] ] ] ] '}' ] ] ]
```

R29 Expression

```
a OR b
```

Leftmost derivation

```
Expression
=> Conjunction 'OR' Conjunction
=> Negation 'OR' Conjunction
=> Equality 'OR' Conjunction
=> Comparison 'OR' Conjunction
=> Sum 'OR' Conjunction
=> Product 'OR' Conjunction
```

```
=> Unary 'OR' Conjunction
=> Power 'OR' Conjunction
=> Primary 'OR' Conjunction
=> Reference 'OR' Conjunction
=> IDENT:a 'OR' Conjunction
=> IDENT:a 'OR' Negation
=> IDENT:a 'OR' Equality
=> IDENT:a 'OR' Comparison
=> IDENT:a 'OR' Sum
=> IDENT:a 'OR' Product
=> IDENT:a 'OR' Unary
=> IDENT:a 'OR' Power
=> IDENT:a 'OR' Primary
=> IDENT:a 'OR' Reference
=> IDENT:a 'OR' IDENT:b
```

Rightmost derivation

```
Expression
=> Conjunction 'OR' Conjunction
=> Conjunction 'OR' Negation
=> Conjunction 'OR' Equality
=> Conjunction 'OR' Comparison
=> Conjunction 'OR' Sum
=> Conjunction 'OR' Product
=> Conjunction 'OR' Unary
=> Conjunction 'OR' Power
=> Conjunction 'OR' Primary
=> Conjunction 'OR' Reference
=> Conjunction 'OR' IDENT:b
=> Negation 'OR' IDENT:b
=> Equality 'OR' IDENT:b
=> Comparison 'OR' IDENT:b
=> Sum 'OR' IDENT:b
=> Product 'OR' IDENT:b
=> Unary 'OR' IDENT:b
=> Power 'OR' IDENT:b
=> Primary 'OR' IDENT:b
=> Reference 'OR' IDENT:b
=> IDENT:a 'OR' IDENT:b
```

Parse tree

```
Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[
   Reference[ IDENT:a ] ] ] ] ] ] ] ] ] ] 'OR' Conjunction[ Negation[ Equality[ Comparison[ Sum[
   Product[ Unary[ Power[ Primary[ Reference[ IDENT:b ] ] ] ] ] ] ] ] ] ] ]
```

R30 Conjunction

```
left AND right
```

Leftmost derivation

```
Conjunction
=> Negation 'AND' Negation
=> Equality 'AND' Negation
=> Comparison 'AND' Negation
=> Sum 'AND' Negation
=> Product 'AND' Negation
=> Unary 'AND' Negation
=> Power 'AND' Negation
=> Primary 'AND' Negation
=> Reference 'AND' Negation
=> IDENT:left 'AND' Negation
=> IDENT:left 'AND' Equality
=> IDENT:left 'AND' Comparison
=> IDENT:left 'AND' Sum
=> IDENT:left 'AND' Product
=> IDENT:left 'AND' Unary
=> IDENT:left 'AND' Power
=> IDENT:left 'AND' Primary
=> IDENT:left 'AND' Reference
=> IDENT:left 'AND' IDENT:right
```

Rightmost derivation

```
Conjunction
=> Negation 'AND' Negation
=> Negation 'AND' Equality
=> Negation 'AND' Comparison
=> Negation 'AND' Sum
=> Negation 'AND' Product
=> Negation 'AND' Unary
=> Negation 'AND' Power
=> Negation 'AND' Primary
=> Negation 'AND' Reference
=> Negation 'AND' IDENT:right
=> Equality 'AND' IDENT:right
=> Comparison 'AND' IDENT:right
=> Sum 'AND' IDENT:right
=> Product 'AND' IDENT:right
=> Unary 'AND' IDENT:right
=> Power 'AND' IDENT:right
=> Primary 'AND' IDENT:right
=> Reference 'AND' IDENT:right
=> IDENT:left 'AND' IDENT:right
```

Parse tree

```
Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[
   IDENT:left ] ] ] ] ] ] ] ] ] 'AND' Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[
   Primary[ Reference[ IDENT:right ] ] ] ] ] ] ] ] ] ]
```

R31 Negation

```
NOT found
```

Leftmost derivation

```
Negation
=> 'NOT' Negation
=> 'NOT' Equality
=> 'NOT' Comparison
=> 'NOT' Sum
=> 'NOT' Product
=> 'NOT' Unary
=> 'NOT' Power
=> 'NOT' Primary
=> 'NOT' Reference
=> 'NOT' IDENT:found
```

Rightmost derivation

```
Negation
=> 'NOT' Negation
=> 'NOT' Equality
=> 'NOT' Comparison
=> 'NOT' Sum
=> 'NOT' Product
=> 'NOT' Unary
=> 'NOT' Power
=> 'NOT' Primary
=> 'NOT' Reference
=> 'NOT' IDENT:found
```

Parse tree

```
Negation[ 'NOT' Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[
   IDENT:found ] ] ] ] ] ] ] ] ]
```

R32 Equality

```
mid == target
```

Leftmost derivation

```
Equality
=> Comparison '==' Comparison
=> Sum '==' Comparison
=> Product '==' Comparison
=> Unary '==' Comparison
=> Power '==' Comparison
=> Primary '==' Comparison
=> Reference '==' Comparison
=> IDENT:mid '==' Comparison
=> IDENT:mid '==' Sum
=> IDENT:mid '==' Product
=> IDENT:mid '==' Unary
=> IDENT:mid '==' Power
=> IDENT:mid '==' Primary
=> IDENT:mid '==' Reference
=> IDENT:mid '==' IDENT:target
```

Rightmost derivation

```
Equality
=> Comparison '==' Comparison
=> Comparison '==' Sum
=> Comparison '==' Product
=> Comparison '==' Unary
=> Comparison '==' Power
=> Comparison '==' Primary
=> Comparison '==' Reference
=> Comparison '==' IDENT:target
=> Sum '==' IDENT:target
=> Product '==' IDENT:target
=> Unary '==' IDENT:target
=> Power '==' IDENT:target
=> Primary '==' IDENT:target
=> Reference '==' IDENT:target
=> IDENT:mid '==' IDENT:target
```

Parse tree

```
Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:mid ] ] ] ] ] ] ] '=='
   Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:target ] ] ] ] ] ] ] ]
```

R33 Comparison

```
low < high
```

Leftmost derivation

```
Comparison
=> Sum '<' Sum
=> Product '<' Sum
=> Unary '<' Sum
=> Power '<' Sum
=> Primary '<' Sum
=> Reference '<' Sum
=> IDENT:low '<' Sum
=> IDENT:low '<' Product
=> IDENT:low '<' Unary
=> IDENT:low '<' Power
=> IDENT:low '<' Primary
=> IDENT:low '<' Reference
=> IDENT:low '<' IDENT:high
```

Rightmost derivation

```
Comparison
=> Sum '<' Sum
=> Sum '<' Product
=> Sum '<' Unary
=> Sum '<' Power
=> Sum '<' Primary
```

```
=> Sum '<' Reference
=> Sum '<' IDENT:high
=> Product '<' IDENT:high
=> Unary '<' IDENT:high
=> Power '<' IDENT:high
=> Primary '<' IDENT:high
=> Reference '<' IDENT:high
=> IDENT:low '<' IDENT:high
```

Parse tree

```
Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:low ] ] ] ] ] ] '<' Sum[ Product[
   Unary[ Power[ Primary[ Reference[ IDENT:high ] ] ] ] ] ] ]
```

R34 Sum

```
a + b
```

Leftmost derivation

```
Sum
=> Product '+' Product
=> Unary '+' Product
=> Power '+' Product
=> Primary '+' Product
=> Reference '+' Product
=> IDENT:a '+' Product
=> IDENT:a '+' Unary
=> IDENT:a '+' Power
=> IDENT:a '+' Primary
=> IDENT:a '+' Reference
=> IDENT:a '+' IDENT:b
```

Rightmost derivation

```
Sum
=> Product '+' Product
=> Product '+' Unary
=> Product '+' Power
=> Product '+' Primary
=> Product '+' Reference
=> Product '+' IDENT:b
=> Unary '+' IDENT:b
=> Power '+' IDENT:b
=> Primary '+' IDENT:b
=> Reference '+' IDENT:b
=> IDENT:a '+' IDENT:b
```

Parse tree

```
Sum[ Product[ Unary[ Power[ Primary[ Reference[ IDENT:a ] ] ] ] ] '+' Product[ Unary[ Power[
   Primary[ Reference[ IDENT:b ] ] ] ] ] ]
```

R35 Product

```
n DIV 2
```

Leftmost derivation

```
Product
=> Unary 'DIV' Unary
=> Power 'DIV' Unary
=> Primary 'DIV' Unary
=> Reference 'DIV' Unary
=> IDENT:n 'DIV' Unary
=> IDENT:n 'DIV' Power
=> IDENT:n 'DIV' Primary
=> IDENT:n 'DIV' Literal
=> IDENT:n 'DIV' INT_LIT:2
```

Rightmost derivation

```
Product
=> Unary 'DIV' Unary
=> Unary 'DIV' Power
=> Unary 'DIV' Primary
=> Unary 'DIV' Literal
=> Unary 'DIV' INT_LIT:2
=> Power 'DIV' INT_LIT:2
=> Primary 'DIV' INT_LIT:2
=> Reference 'DIV' INT_LIT:2
=> IDENT:n 'DIV' INT_LIT:2
```

Parse tree

```
Product[ Unary[ Power[ Primary[ Reference[ IDENT:n ] ] ] ] 'DIV' Unary[ Power[ Primary[ Literal[
   INT_LIT:2 ] ] ] ] ]
```

R36 `Unary`

```
-2
```

Leftmost derivation

```
Unary
=> '-' Unary
=> '-' Power
=> '-' Primary
=> '-' Literal
=> '-' INT_LIT:2
```

Rightmost derivation

```
Unary
=> '-' Unary
=> '-' Power
=> '-' Primary
=> '-' Literal
=> '-' INT_LIT:2
```

Parse tree

```
Unary[ '-' Unary[ Power[ Primary[ Literal[ INT_LIT:2 ] ] ] ] ]
```

R37 `Power`

```
2 ^ 3 ^ 2
```

Leftmost derivation

```
Power
=> Primary '^' Unary
=> Literal '^' Unary
=> INT_LIT:2 '^' Unary
=> INT_LIT:2 '^' Power
=> INT_LIT:2 '^' Primary '^' Unary
=> INT_LIT:2 '^' Literal '^' Unary
=> INT_LIT:2 '^' INT_LIT:3 '^' Unary
=> INT_LIT:2 '^' INT_LIT:3 '^' Power
=> INT_LIT:2 '^' INT_LIT:3 '^' Primary
=> INT_LIT:2 '^' INT_LIT:3 '^' Literal
=> INT_LIT:2 '^' INT_LIT:3 '^' INT_LIT:2
```

Rightmost derivation

```
Power
=> Primary '^' Unary
=> Primary '^' Power
=> Primary '^' Primary '^' Unary
=> Primary '^' Primary '^' Power
=> Primary '^' Primary '^' Primary
=> Primary '^' Primary '^' Literal
=> Primary '^' Primary '^' INT_LIT:2
=> Primary '^' Literal '^' INT_LIT:2
=> Primary '^' INT_LIT:3 '^' INT_LIT:2
=> Literal '^' INT_LIT:3 '^' INT_LIT:2
```

```
=> INT_LIT:2 '^' INT_LIT:3 '^' INT_LIT:2
```

Parse tree

```
Power[ Primary[ Literal[ INT_LIT:2 ] ] '^' Unary[ Power[ Primary[ Literal[ INT_LIT:3 ] ] '^' Unary[
   Power[ Primary[ Literal[ INT_LIT:2 ] ] ] ] ] ] ]
```

R38 `Primary`

```
(value)
```

Leftmost derivation

```
Primary
=> '(' Expression ')'
=> '(' Conjunction ')'
=> '(' Negation ')'
=> '(' Equality ')'
=> '(' Comparison ')'
=> '(' Sum ')'
=> '(' Product ')'
=> '(' Unary ')'
=> '(' Power ')'
=> '(' Primary ')'
=> '(' Reference ')'
=> '(' IDENT:value ')'
```

Rightmost derivation

```
Primary
=> '(' Expression ')'
=> '(' Conjunction ')'
=> '(' Negation ')'
=> '(' Equality ')'
=> '(' Comparison ')'
=> '(' Sum ')'
=> '(' Product ')'
=> '(' Unary ')'
=> '(' Power ')'
=> '(' Primary ')'
=> '(' Reference ')'
=> '(' IDENT:value ')'
```

Parse tree

```
Primary[ '(' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[ Power[
   Primary[ Reference[ IDENT:value ] ] ] ] ] ] ] ] ] ] ] ')' ]
```

R39 `Reference`

```
numbers[mid]
```

Leftmost derivation

```
Reference
=> IDENT:numbers '[' Expression ']'
=> IDENT:numbers '[' Conjunction ']'
=> IDENT:numbers '[' Negation ']'
=> IDENT:numbers '[' Equality ']'
=> IDENT:numbers '[' Comparison ']'
=> IDENT:numbers '[' Sum ']'
=> IDENT:numbers '[' Product ']'
=> IDENT:numbers '[' Unary ']'
=> IDENT:numbers '[' Power ']'
=> IDENT:numbers '[' Primary ']'
=> IDENT:numbers '[' Reference ']'
=> IDENT:numbers '[' IDENT:mid ']'
```

Rightmost derivation

```
Reference
=> IDENT:numbers '[' Expression ']'
=> IDENT:numbers '[' Conjunction ']'
=> IDENT:numbers '[' Negation ']'
=> IDENT:numbers '[' Equality ']'
```

```
=> IDENT:numbers '[' Comparison ']'
=> IDENT:numbers '[' Sum ']'
=> IDENT:numbers '[' Product ']'
=> IDENT:numbers '[' Unary ']'
=> IDENT:numbers '[' Power ']'
=> IDENT:numbers '[' Primary ']'
=> IDENT:numbers '[' Reference ']'
=> IDENT:numbers '[' IDENT:mid ']'
```

Parse tree

```
Reference[ IDENT:numbers '[' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[
   Unary[ Power[ Primary[ Reference[ IDENT:mid ] ] ] ] ] ] ] ] ] ] ] ']' ]
```

R40 `Literal`

```
'A'
```

Leftmost derivation

```
Literal
=> CHAR_LIT:'A'
```

Rightmost derivation

```
Literal
=> CHAR_LIT:'A'
```

Parse tree

```
Literal[ CHAR_LIT:'A' ]
```

R41 `ArrayLiteral`

```
[2, 4]
```

Leftmost derivation

```
ArrayLiteral
=> '[' Expression ',' Expression ']'
=> '[' Conjunction ',' Expression ']'
=> '[' Negation ',' Expression ']'
=> '[' Equality ',' Expression ']'
=> '[' Comparison ',' Expression ']'
=> '[' Sum ',' Expression ']'
=> '[' Product ',' Expression ']'
=> '[' Unary ',' Expression ']'
=> '[' Power ',' Expression ']'
=> '[' Primary ',' Expression ']'
=> '[' Literal ',' Expression ']'
=> '[' INT_LIT:2 ',' Expression ']'
=> '[' INT_LIT:2 ',' Conjunction ']'
=> '[' INT_LIT:2 ',' Negation ']'
=> '[' INT_LIT:2 ',' Equality ']'
=> '[' INT_LIT:2 ',' Comparison ']'
=> '[' INT_LIT:2 ',' Sum ']'
=> '[' INT_LIT:2 ',' Product ']'
=> '[' INT_LIT:2 ',' Unary ']'
=> '[' INT_LIT:2 ',' Power ']'
=> '[' INT_LIT:2 ',' Primary ']'
=> '[' INT_LIT:2 ',' Literal ']'
=> '[' INT_LIT:2 ',' INT_LIT:4 ']'
```

Rightmost derivation

```
ArrayLiteral
=> '[' Expression ',' Expression ']'
=> '[' Expression ',' Conjunction ']'
=> '[' Expression ',' Negation ']'
=> '[' Expression ',' Equality ']'
=> '[' Expression ',' Comparison ']'
=> '[' Expression ',' Sum ']'
=> '[' Expression ',' Product ']'
=> '[' Expression ',' Unary ']'
=> '[' Expression ',' Power ']'
```

```
=> '[' Expression ',' Primary ']'
=> '[' Expression ',' Literal ']'
=> '[' Expression ',' INT_LIT:4 ']'
=> '[' Conjunction ',' INT_LIT:4 ']'
=> '[' Negation ',' INT_LIT:4 ']'
=> '[' Equality ',' INT_LIT:4 ']'
=> '[' Comparison ',' INT_LIT:4 ']'
=> '[' Sum ',' INT_LIT:4 ']'
=> '[' Product ',' INT_LIT:4 ']'
=> '[' Unary ',' INT_LIT:4 ']'
=> '[' Power ',' INT_LIT:4 ']'
=> '[' Primary ',' INT_LIT:4 ']'
=> '[' Literal ',' INT_LIT:4 ']'
=> '[' INT_LIT:2 ',' INT_LIT:4 ']'
```

Parse tree

```
ArrayLiteral[ '[' Expression[ Conjunction[ Negation[ Equality[ Comparison[ Sum[ Product[ Unary[
   Power[ Primary[ Literal[ INT_LIT:2 ] ] ] ] ] ] ] ] ] ] ] ',' Expression[ Conjunction[ Negation[
   Equality[ Comparison[ Sum[ Product[ Unary[ Power[ Primary[ Literal[ INT_LIT:4 ] ] ] ] ] ] ] ] ] ]
   ] ']' ]
```

A Worked Binary-Search Token Stream

This is the complete lexical output for the 20-line lexer regression fixture `sample.lal`, counting `ARRAY numbers` as line 1. Each indentation level uses four spaces. The closing brace on line 20 is followed by one line feed; EOF is therefore at line 21, column 1. The comment on line 9 is included as a LINE_COMMENT entry; its displayed text is shortened here to match the console-width limit. Positions, source lexemes, and newline flags are retained; every `suspectedKeyword` flag is **FALSE**.

The fixture predates the syntax proposal and omits semicolons on simple statements. Chapter 2 shows the updated binary-search source. Read the lexical rows from top to bottom. The last column is `precededByNewline`; an empty lexeme belongs only to EOF.

Line:column	Type	Lexeme	Newline before
1:1	KW_ARRAY	ARRAY	FALSE
1:7	IDENT	numbers	FALSE
1:15	ASSIGN	=	FALSE
1:17	LBRACKET	[	FALSE
1:18	INTEGER	2	FALSE
1:19	COMMA	,	FALSE
1:21	INTEGER	4	FALSE
1:22	COMMA	,	FALSE
1:24	INTEGER	6	FALSE
1:25	COMMA	,	FALSE
1:27	INTEGER	8	FALSE
1:28	COMMA	,	FALSE
1:30	INTEGER	10	FALSE
1:32	RBRACKET	]	FALSE
2:1	KW_INTEGER	INTEGER	TRUE
2:9	IDENT	target	FALSE
2:16	ASSIGN	=	FALSE
2:18	INTEGER	8	FALSE
3:1	KW_INTEGER	INTEGER	TRUE
3:9	IDENT	low	FALSE
3:13	ASSIGN	=	FALSE
3:15	INTEGER	0	FALSE
4:1	KW_INTEGER	INTEGER	TRUE
4:9	IDENT	high	FALSE
4:14	ASSIGN	=	FALSE
4:16	INTEGER	4	FALSE
5:1	KW_INTEGER	INTEGER	TRUE
5:9	IDENT	mid	FALSE
5:13	ASSIGN	=	FALSE
5:15	INTEGER	0	FALSE
6:1	KW_BOOLEAN	BOOLEAN	TRUE
6:9	IDENT	found	FALSE
6:15	ASSIGN	=	FALSE
6:17	FALSE	FALSE	FALSE
7:1	KW_WHILE	WHILE	TRUE
7:7	LPAREN	(	FALSE
7:8	IDENT	low	FALSE
7:12	LE	<=	FALSE
7:15	IDENT	high	FALSE
7:20	AND	AND	FALSE
7:24	IDENT	found	FALSE
7:30	EQ	==	FALSE
7:33	FALSE	FALSE	FALSE
7:38	RPAREN	)	FALSE
7:40	LBRACE	{	FALSE

Continued on next page

Line:column	Type	Lexeme	Newline before
8:5	IDENT	mid	TRUE
8:9	ASSIGN	=	FALSE
8:11	IDENT	low	FALSE
8:15	PLUS	+	FALSE
8:17	LPAREN	(	FALSE
8:18	IDENT	high	FALSE
8:23	MINUS	-	FALSE
8:25	IDENT	low	FALSE
8:28	RPAREN	)	FALSE
8:30	DIV	DIV	FALSE
8:34	INTEGER	2	FALSE
9:5	LINE_COMMENT	// Update search boun...	TRUE
10:5	KW_IF	IF	TRUE
10:8	LPAREN	(	FALSE
10:9	IDENT	numbers	FALSE
10:16	LBRACKET	[	FALSE
10:17	IDENT	mid	FALSE
10:20	RBRACKET	]	FALSE
10:22	EQ	==	FALSE
10:25	IDENT	target	FALSE
10:31	RPAREN	)	FALSE
10:33	LBRACE	{	FALSE
11:9	IDENT	found	TRUE
11:15	ASSIGN	=	FALSE
11:17	TRUE	TRUE	FALSE
12:9	KW_OUTPUT	OUTPUT	TRUE
12:16	IDENT	mid	FALSE
13:5	RBRACE	}	TRUE
14:5	KW_ELSE	ELSE	TRUE
14:10	KW_IF	IF	FALSE
14:13	LPAREN	(	FALSE
14:14	IDENT	numbers	FALSE
14:21	LBRACKET	[	FALSE
14:22	IDENT	mid	FALSE
14:25	RBRACKET	]	FALSE
14:27	LT	<	FALSE
14:29	IDENT	target	FALSE
14:35	RPAREN	)	FALSE
14:37	LBRACE	{	FALSE
15:9	IDENT	low	TRUE
15:13	ASSIGN	=	FALSE
15:15	IDENT	mid	FALSE
15:19	PLUS	+	FALSE
15:21	INTEGER	1	FALSE
16:5	RBRACE	}	TRUE
17:5	KW_ELSE	ELSE	TRUE
17:10	LBRACE	{	FALSE
18:9	IDENT	high	TRUE
18:14	ASSIGN	=	FALSE
18:16	IDENT	mid	FALSE
18:20	MINUS	-	FALSE
18:22	INTEGER	1	FALSE
19:5	RBRACE	}	TRUE
20:1	RBRACE	}	TRUE
21:1	EOF		TRUE

B Complete Unified DFA Transition Specification

This appendix exhaustively specifies the transition table built by `lalgo_unified.c`: 254 explicit states and 83 input classes, including virtual EOF. It is generated from the implementation during the paper build. The overview in Section 2.3.1 and the smaller category diagrams are readable projections; the tables here supply every concrete state and defined transition.

Each uppercase and lowercase letter is a distinct class (52 classes). DIGIT denotes 0–9. The next 27 classes identify underscore, dot, assignment/equality, exclamation mark, comparisons, slashes, quotes, backslash, whitespace, arithmetic symbols, brackets and punctuation, in their named order. OTHER denotes remaining printable ASCII; ILLEGAL denotes unsupported bytes; EOF is virtual. The alphabet mapping follows the character-set section. All omitted entries are $\delta(q, c) = q_\perp$; the implicit non-accepting sink has $\delta(q_\perp, c) = q_\perp$ for every class. Thus the mathematical transition function is total, although the implementation stores missing transitions as -1 and stops lookahead there.

NONE means non-accepting; SKIP is an accepting whitespace action; every token name labels an accepting state, including INVALID for malformed-number candidates. “Suspected” sets the capitalization diagnostic. “Committed” requires an error when scanning stops before a closing quote or block-comment terminator, even if a shorter prefix was accepted. EOF is emitted by the scanner when the source is exhausted, not by an accepting DFA state. Token emission and restarting at q_0 are scanner actions, not epsilon transitions.

B.1 All Transitions from the Single Start State

Destination	Input classes from q_0
q_{62}	A
q_{48}	B
q_{30}	C
q_{16}	D
q_{86}	E
q_{104}	F
q_{1}	G, H, J, K, L, M, P, Q, U, V, X, Y, Z, g, h, j, k, l, m, p, q, u, v, x, y, z
q_{2}	I
q_{154}	N
q_{116}	O
q_{176}	R
q_{72}	S
q_{128}	T
q_{94}	W
q_{63}	a
q_{49}	b
q_{31}	c
q_{17}	d
q_{87}	e
q_{105}	f
q_{3}	i
q_{155}	n
q_{117}	o
q_{177}	r
q_{73}	s
q_{129}	t
q_{95}	w
q_{216}	DIGIT
q_{239}	DOT
q_{220}	EQ

Destination	Input classes from q_0
q_{222}	BANG
q_{224}	LT
q_{226}	GT
q_{228}	SLASH
q_{242}	STAR
q_{233}	DQUOTE
q_{235}	SQUOTE
q_{238}	NEWLINE, SPACE, TAB
q_{240}	PLUS
q_{241}	MINUS
q_{243}	PERCENT
q_{244}	CARET
q_{245}	LPAREN
q_{246}	RPAREN
q_{247}	LBRACKET
q_{248}	RBRACKET
q_{249}	LBRACE
q_{250}	RBRACE
q_{251}	COMMA
q_{252}	SEMICOLON
q_{253}	COLON

B.2 All States, Accepting Actions, and Remaining Transitions

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_0 `START`	NONE	See the start-state table above.
q_1 `ID`	IDENT	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_2 `E:I`	IDENT	A–Z except F,N, a–z except f,n, DIGIT, UNDERSCORE $\rightarrow q_1$; F $\rightarrow q_{84}$; N $\rightarrow q_4$; f $\rightarrow q_{85}$; n $\rightarrow q_5$
q_3 `F:I`	IDENT	A–Z except F,N, a–z except f,n, DIGIT, UNDERSCORE $\rightarrow q_1$; F, f $\rightarrow q_{85}$; N, n $\rightarrow q_5$
q_4 `E:IN`	IDENT	A–Z except P,T,V, a–z except p,t,v, DIGIT, UNDERSCORE $\rightarrow q_1$; P $\rightarrow q_{110}$; T $\rightarrow q_6$; V $\rightarrow q_{194}$; p $\rightarrow q_{111}$; t $\rightarrow q_7$; v $\rightarrow q_{195}$
q_5 `F:IN`	IDENT	A–Z except P,T,V, a–z except p,t,v, DIGIT, UNDERSCORE $\rightarrow q_1$; P, p $\rightarrow q_{111}$; T, t $\rightarrow q_7$; V, v $\rightarrow q_{195}$
q_6 `E:INT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_8$; e $\rightarrow q_9$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_7 `F:INT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_9
q_8 `E:INTE`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE → q_1; G → q_{10}; g → q_{11}
q_9 `F:INTE`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE → q_1; G, g → q_{11}
q_{10} `E:INTEG`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{12}; e → q_{13}
q_{11} `F:INTEG`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{13}
q_{12} `E:INTEGE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{14}; r → q_{15}
q_{13} `F:INTEGE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{15}
q_{14} `E:INTEGER`	KW_INTEGER	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{15} `F:INTEGER`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{16} `E:D`	IDENT	A–Z except E,I, a–z except e,i, DIGIT, UNDERSCORE → q_1; E → q_{18}; I → q_{144}; e → q_{19}; i → q_{145}
q_{17} `F:D`	IDENT	A–Z except E,I, a–z except e,i, DIGIT, UNDERSCORE → q_1; E, e → q_{19}; I, i → q_{145}
q_{18} `E:DE`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{20}; c → q_{21}
q_{19} `F:DE`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{21}
q_{20} `E:DEC`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{22}; i → q_{23}
q_{21} `F:DEC`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{23}
q_{22} `E:DECI`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M → q_{24}; m → q_{25}
q_{23} `F:DECI`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M, m → q_{25}
q_{24} `E:DECIM`	IDENT	A → q_{26}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{27}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{25} `F:DECIM`	IDENT	A, a → q_{27}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{26} `E:DECIMA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{28}; l → q_{29}
q_{27} `F:DECIMA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{29}
q_{28} `E:DECIMAL`	KW_DECIMAL	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{29} `F:DECIMAL`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{30} `E:C`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H → q_{32}; h → q_{33}
q_{31} `F:C`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H, h → q_{33}
q_{32} `E:CH`	IDENT	A → q_{34}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{35}
q_{33} `F:CH`	IDENT	A, a → q_{35}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{34} `E:CHA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{36}; r → q_{37}
q_{35} `F:CHA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{37}
q_{36} `E:CHAR`	IDENT	A → q_{38}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{39}
q_{37} `F:CHAR`	IDENT	A, a → q_{39}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{38} `E:CHARA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{40}; c → q_{41}
q_{39} `F:CHARA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{41}
q_{40} `E:CHARAC`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{42}; t → q_{43}
q_{41} `F:CHARAC`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{43}
q_{42} `E:CHARACT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{44}; e → q_{45}
q_{43} `F:CHARACT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{45}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{44} `E:CHARACTE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{46}; r → q_{47}
q_{45} `F:CHARACTE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{47}
q_{46} `E:CHARACTER`	KW_CHARACTER	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{47} `F:CHARACTER`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{48} `E:B`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O → q_{50}; o → q_{51}
q_{49} `F:B`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{51}
q_{50} `E:BO`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O → q_{52}; o → q_{53}
q_{51} `F:BO`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{53}
q_{52} `E:BOO`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{54}; l → q_{55}
q_{53} `F:BOO`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{55}
q_{54} `E:BOOL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{56}; e → q_{57}
q_{55} `F:BOOL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{57}
q_{56} `E:BOOLE`	IDENT	A → q_{58}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{59}
q_{57} `F:BOOLE`	IDENT	A, a → q_{59}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{58} `E:BOOLEA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{60}; n → q_{61}
q_{59} `F:BOOLEA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{61}
q_{60} `E:BOOLEAN`	KW_BOOLEAN	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{61} `F:BOOLEAN`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{62} `E:A`	IDENT	A–Z except L,N,R, a–z except l,n,r, DIGIT, UNDERSCORE → q_1; L → q_{160}; N → q_{148}; R → q_{64}; l → q_{161}; n → q_{149}; r → q_{65}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{63} `F:A`	IDENT	A–Z except L,N,R, a–z except l,n,r, DIGIT, UNDERSCORE → q_1; L, l → q_{161}; N, n → q_{149}; R, r → q_{65}
q_{64} `E:AR`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{66}; r → q_{67}
q_{65} `F:AR`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{67}
q_{66} `E:ARR`	IDENT	A → q_{68}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{69}
q_{67} `F:ARR`	IDENT	A, a → q_{69}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{68} `E:ARRA`	IDENT	A–Z except Y, a–z except y, DIGIT, UNDERSCORE → q_1; Y → q_{70}; y → q_{71}
q_{69} `F:ARRA`	IDENT	A–Z except Y, a–z except y, DIGIT, UNDERSCORE → q_1; Y, y → q_{71}
q_{70} `E:ARRAY`	KW_ARRAY	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{71} `F:ARRAY`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{72} `E:S`	IDENT	A–Z except E,T, a–z except e,t, DIGIT, UNDERSCORE → q_1; E → q_{208}; T → q_{74}; e → q_{209}; t → q_{75}
q_{73} `F:S`	IDENT	A–Z except E,T, a–z except e,t, DIGIT, UNDERSCORE → q_1; E, e → q_{209}; T, t → q_{75}
q_{74} `E:ST`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{76}; r → q_{77}
q_{75} `F:ST`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{77}
q_{76} `E:STR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{78}; i → q_{79}
q_{77} `F:STR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{79}
q_{78} `E:STRI`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{80}; n → q_{81}
q_{79} `F:STRI`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{81}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{80} `E:STRIN`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G $\rightarrow q_{82}$; g $\rightarrow q_{83}$
q_{81} `F:STRIN`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G, g $\rightarrow q_{83}$
q_{82} `E:STRING`	KW_STRING	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{83} `F:STRING`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{84} `E:IF`	KW_IF	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{85} `F:IF`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{86} `E:E`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L $\rightarrow q_{88}$; l $\rightarrow q_{89}$
q_{87} `F:E`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L, l $\rightarrow q_{89}$
q_{88} `E:EL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S $\rightarrow q_{90}$; s $\rightarrow q_{91}$
q_{89} `F:EL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S, s $\rightarrow q_{91}$
q_{90} `E:ELS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{92}$; e $\rightarrow q_{93}$
q_{91} `F:ELS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E, e $\rightarrow q_{93}$
q_{92} `E:ELSE`	KW_ELSE	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{93} `F:ELSE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{94} `E:W`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE $\rightarrow q_1$; H $\rightarrow q_{96}$; h $\rightarrow q_{97}$
q_{95} `F:W`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE $\rightarrow q_1$; H, h $\rightarrow q_{97}$
q_{96} `E:WH`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE $\rightarrow q_1$; I $\rightarrow q_{98}$; i $\rightarrow q_{99}$
q_{97} `F:WH`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE $\rightarrow q_1$; I, i $\rightarrow q_{99}$
q_{98} `E:WHI`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L $\rightarrow q_{100}$; l $\rightarrow q_{101}$
q_{99} `F:WHI`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L, l $\rightarrow q_{101}$
q_{100} `E:WHIL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{102}$; e $\rightarrow q_{103}$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{101} `F:WHIL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{103}
q_{102} `E:WHILE`	KW_WHILE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{103} `F:WHILE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{104} `E:F`	IDENT	A → q_{136}; A–Z except A,O, a–z except a,o, DIGIT, UNDERSCORE → q_1; O → q_{106}; a → q_{137}; o → q_{107}
q_{105} `F:F`	IDENT	A, a → q_{137}; A–Z except A,O, a–z except a,o, DIGIT, UNDERSCORE → q_1; O, o → q_{107}
q_{106} `E:FO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{108}; r → q_{109}
q_{107} `F:FO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{109}
q_{108} `E:FOR`	KW_FOR	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{109} `F:FOR`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{110} `E:INP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{112}; u → q_{113}
q_{111} `F:INP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{113}
q_{112} `E:INPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{114}; t → q_{115}
q_{113} `F:INPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{115}
q_{114} `E:INPUT`	KW_INPUT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{115} `F:INPUT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{116} `E:O`	IDENT	A–Z except R,U, a–z except r,u, DIGIT, UNDERSCORE → q_1; R → q_{152}; U → q_{118}; r → q_{153}; u → q_{119}
q_{117} `F:O`	IDENT	A–Z except R,U, a–z except r,u, DIGIT, UNDERSCORE → q_1; R, r → q_{153}; U, u → q_{119}
q_{118} `E:OU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{120}; t → q_{121}
q_{119} `F:OU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{121}
q_{120} `E:OUT`	IDENT	A–Z except P, a–z except p, DIGIT, UNDERSCORE → q_1; P → q_{122}; p → q_{123}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{121} `F:OUT`	IDENT	A–Z except P, a–z except p, DIGIT, UNDERSCORE → q_1; P, p → q_{123}
q_{122} `E:OUTP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{124}; u → q_{125}
q_{123} `F:OUTP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{125}
q_{124} `E:OUTPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{126}; t → q_{127}
q_{125} `F:OUTPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{127}
q_{126} `E:OUTPUT`	KW_OUTPUT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{127} `F:OUTPUT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{128} `E:T`	IDENT	A–Z except H,R, a–z except h,r, DIGIT, UNDERSCORE → q_1; H → q_{212}; R → q_{130}; h → q_{213}; r → q_{131}
q_{129} `F:T`	IDENT	A–Z except H,R, a–z except h,r, DIGIT, UNDERSCORE → q_1; H, h → q_{213}; R, r → q_{131}
q_{130} `E:TR`	IDENT	A → q_{188}; A–Z except A,U, a–z except a,u, DIGIT, UNDERSCORE → q_1; U → q_{132}; a → q_{189}; u → q_{133}
q_{131} `F:TR`	IDENT	A, a → q_{189}; A–Z except A,U, a–z except a,u, DIGIT, UNDERSCORE → q_1; U, u → q_{133}
q_{132} `E:TRU`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{134}; e → q_{135}
q_{133} `F:TRU`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{135}
q_{134} `E:TRUE`	TRUE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{135} `F:TRUE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{136} `E:FA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{138}; l → q_{139}
q_{137} `F:FA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{139}
q_{138} `E:FAL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE → q_1; S → q_{140}; s → q_{141}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{139} `F:FAL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S, s $\rightarrow q_{141}$
q_{140} `E:FALS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{142}$; e $\rightarrow q_{143}$
q_{141} `F:FALS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E, e $\rightarrow q_{143}$
q_{142} `E:FALSE`	FALSE	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{143} `F:FALSE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{144} `E:DI`	IDENT	A–Z except V, a–z except v, DIGIT, UNDERSCORE $\rightarrow q_1$; V $\rightarrow q_{146}$; v $\rightarrow q_{147}$
q_{145} `F:DI`	IDENT	A–Z except V, a–z except v, DIGIT, UNDERSCORE $\rightarrow q_1$; V, v $\rightarrow q_{147}$
q_{146} `E:DIV`	DIV	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{147} `F:DIV`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{148} `E:AN`	IDENT	A–Z except D, a–z except d, DIGIT, UNDERSCORE $\rightarrow q_1$; D $\rightarrow q_{150}$; d $\rightarrow q_{151}$
q_{149} `F:AN`	IDENT	A–Z except D, a–z except d, DIGIT, UNDERSCORE $\rightarrow q_1$; D, d $\rightarrow q_{151}$
q_{150} `E:AND`	AND	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{151} `F:AND`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{152} `E:OR`	OR	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{153} `F:OR`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{154} `E:N`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O $\rightarrow q_{156}$; o $\rightarrow q_{157}$
q_{155} `F:N`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O, o $\rightarrow q_{157}$
q_{156} `E:NO`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE $\rightarrow q_1$; T $\rightarrow q_{158}$; t $\rightarrow q_{159}$
q_{157} `F:NO`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE $\rightarrow q_1$; T, t $\rightarrow q_{159}$
q_{158} `E:NOT`	NOT	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{159} `F:NOT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{160} `E:AL`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G $\rightarrow q_{162}$; g $\rightarrow q_{163}$
q_{161} `F:AL`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G, g $\rightarrow q_{163}$
q_{162} `E:ALG`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O $\rightarrow q_{164}$; o $\rightarrow q_{165}$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{163} `F:ALG`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{165}
q_{164} `E:ALGO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{166}; r → q_{167}
q_{165} `F:ALGO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{167}
q_{166} `E:ALGOR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{168}; i → q_{169}
q_{167} `F:ALGOR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{169}
q_{168} `E:ALGORI`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{170}; t → q_{171}
q_{169} `F:ALGORI`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{171}
q_{170} `E:ALGORIT`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H → q_{172}; h → q_{173}
q_{171} `F:ALGORIT`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H, h → q_{173}
q_{172} `E:ALGORITH`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M → q_{174}; m → q_{175}
q_{173} `F:ALGORITH`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M, m → q_{175}
q_{174} `E:ALGORITHM`	RES_ALGORITHM	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{175} `F:ALGORITHM`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{176} `E:R`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{178}; e → q_{179}
q_{177} `F:R`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{179}
q_{178} `E:RE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{180}; t → q_{181}
q_{179} `F:RE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{181}
q_{180} `E:RET`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{182}; u → q_{183}
q_{181} `F:RET`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{183}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{182} `E:RETU`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{184}; r → q_{185}
q_{183} `F:RETU`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{185}
q_{184} `E:RETUR`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{186}; n → q_{187}
q_{185} `F:RETUR`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{187}
q_{186} `E:RETURN`	RES_RETURN	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{187} `F:RETURN`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{188} `E:TRA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{190}; c → q_{191}
q_{189} `F:TRA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{191}
q_{190} `E:TRAC`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{192}; e → q_{193}
q_{191} `F:TRAC`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{193}
q_{192} `E:TRACE`	RES_TRACE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{193} `F:TRACE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{194} `E:INV`	IDENT	A → q_{196}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{197}
q_{195} `F:INV`	IDENT	A, a → q_{197}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{196} `E:INVA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{198}; r → q_{199}
q_{197} `F:INVA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{199}
q_{198} `E:INVAR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{200}; i → q_{201}
q_{199} `F:INVAR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{201}
q_{200} `E:INVARI`	IDENT	A → q_{202}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{203}
q_{201} `F:INVARI`	IDENT	A, a → q_{203}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{202} `E:INVARIA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{204}; n → q_{205}
q_{203} `F:INVARIA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{205}
q_{204} `E:INVARIAN`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{206}; t → q_{207}
q_{205} `F:INVARIAN`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{207}
q_{206} `E:INVARIANT`	RES_INVARIANT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{207} `F:INVARIANT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{208} `E:SE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{210}; t → q_{211}
q_{209} `F:SE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{211}
q_{210} `E:SET`	NOISE_SET	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{211} `F:SET`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{212} `E:TH`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{214}; e → q_{215}
q_{213} `F:TH`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{215}
q_{214} `E:THE`	NOISE_THE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{215} `F:THE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{216} `INT`	INTEGER	A–Z, a–z, UNDERSCORE → q_{219}; DIGIT → q_{216}; DOT → q_{217}
q_{217} `INT.`	NONE	DIGIT → q_{218}
q_{218} `DEC`	DECIMAL	A–Z, a–z, UNDERSCORE → q_{219}; DIGIT → q_{218}
q_{219} `BAD_NUM`	INVALID	A–Z, a–z, DIGIT, UNDERSCORE → q_{219}
q_{220} `ASSIGN`	ASSIGN	EQ → q_{221}
q_{221} `EQ`	EQ	None (all inputs stop the match).
q_{222} `BANG`	NONE	EQ → q_{223}
q_{223} `NEQ`	NEQ	None (all inputs stop the match).
q_{224} `LT`	LT	EQ → q_{225}
q_{225} `LE`	LE	None (all inputs stop the match).
q_{226} `GT`	GT	EQ → q_{227}
q_{227} `GE`	GE	None (all inputs stop the match).
q_{228} `SLASH`	SLASH	SLASH → q_{229}; STAR → q_{230}
q_{229} `LINE_CMT`	LINE_COMMENT	all real classes except NEWLINE → q_{229}
q_{230} `BLK_CMT`	NONE; committed comment	all real classes except STAR → q_{230}; STAR → q_{231}
q_{231} `BLK_STAR`	NONE; committed comment	all real classes except SLASH, STAR → q_{230}; SLASH → q_{232}; STAR → q_{231}
q_{232} `BLK_END`	BLOCK_COMMENT	None (all inputs stop the match).
q_{233} `STR_OPEN`	NONE; committed string	all real classes except DQUOTE, BACKSLASH, NEWLINE, ILLEGAL → q_{233}; DQUOTE → q_{234}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{234} `STR_DONE`	STRING	None (all inputs stop the match).
q_{235} `CHR_OPEN`	NONE; committed character	all real classes except SQUOTE, BACKSLASH, NEWLINE, ILLEGAL $\rightarrow q_{236}$
q_{236} `CHR_BODY`	NONE; committed character	SQUOTE $\rightarrow q_{237}$
q_{237} `CHR_DONE`	CHAR	None (all inputs stop the match).
q_{238} `WS`	SKIP	NEWLINE, SPACE, TAB $\rightarrow q_{238}$
q_{239} `DOT`	DOT	None (all inputs stop the match).
q_{240} `PLUS`	PLUS	None (all inputs stop the match).
q_{241} `MINUS`	MINUS	None (all inputs stop the match).
q_{242} `STAR`	STAR	None (all inputs stop the match).
q_{243} `PERCENT`	PERCENT	None (all inputs stop the match).
q_{244} `CARET`	CARET	None (all inputs stop the match).
q_{245} `LPAREN`	LPAREN	None (all inputs stop the match).
q_{246} `RPAREN`	RPAREN	None (all inputs stop the match).
q_{247} `LBRACKET`	LBRACKET	None (all inputs stop the match).
q_{248} `RBRACKET`	RBRACKET	None (all inputs stop the match).
q_{249} `LBRACE`	LBRACE	None (all inputs stop the match).
q_{250} `RBRACE`	RBRACE	None (all inputs stop the match).
q_{251} `COMMA`	COMMA	None (all inputs stop the match).
q_{252} `SEMICOLON`	SEMICOLON	None (all inputs stop the match).
q_{253} `COLON`	COLON	None (all inputs stop the match).